

T.C.
SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ

BSM 498 BİTİRME ÇALIŞMASI

**GÖRME ENGELLİ BİREYLER İÇİN GERÇEK
ZAMANLI GÖRÜNTÜ İŞLEME TABANLI MOBİL
REHBER SİSTEMİ**

B221210025 - Alper ZEYBEK
B221210014 - Mustafa Alperen AKÇA

Fakülte Anabilim Dalı : BİLGİSAYAR MÜHENDİSLİĞİ
Tez Danışmanı : Dr. Öğr. Üyesi Hüseyin ESKİ

2025-2026 Bahar Dönemi

T.C.
SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ

**GÖRME ENGELLİ BİREYLER İÇİN GERÇEK
ZAMANLI GÖRÜNTÜ İŞLEME TABANLI MOBİL
REHBER SİSTEMİ**

BSM 498 - BİTİRME ÇALIŞMASI

B221210025 - Alper ZEYBEK
B221210014 - Mustafa Alperen AKÇA

Fakülte Anabilim Dalı : BİLGİSAYAR MÜHENDİSLİĞİ

Bu tez .. / .. / 2026 tarihinde aşağıdaki jüri tarafından oybirliği / oyçokluğu ile kabul edilmiştir.

.....
Jüri Başkanı

.....
Üye

.....
Üye

ÖNSÖZ

Görme engelli ve az gören bireylerin günlük yaşamda karşılaştıkları zorluklar, yalnızca bireysel hareket kabiliyetini değil, aynı zamanda güvenli yaşam, bağımsızlık ve toplumsal hayata katılım düzeyini de doğrudan etkilemektedir. Özellikle şehir yaşamında düzensiz kaldırımlar, kontrolsüz park edilen araçlar, geçici yol çalışmaları ve çevresel engeller, görme engelli bireyler için ciddi riskler oluşturmakta; mevcut yön bulma uygulamaları bu engellerin büyük bir kısmını algılayamamaktadır. Bu durum, görme engelli bireylerin çevrelerini daha güvenli ve etkin bir biçimde algılayabilmelerine yönelik yeni teknolojik çözümlerin geliştirilmesini gerekli kılmaktadır.

Bu bitirme çalışması, görme engelli ve az gören bireylerin günlük hayatta karşılaştıkları söz konusu problemlere, bilgisayar mühendisliği disiplininin sunduğu çağdaş yazılım ve mobil teknoloji olanaklarıyla çözüm üretme amacıyla gerçekleştirilmiştir. Çalışmanın temel motivasyonu, yalnızca akademik bir gerekliliği yerine getirmek değil; gerçek hayatta karşılığı olan, kullanıcı odaklı ve toplumsal fayda üretmeyi hedefleyen bir sistem ortaya koymaktır. Bu kapsamda, mobil cihazların arka kamerasını kullanan, çevresel nesneleri yapay zekâ tabanlı bir model ile gerçek zamanlı olarak tanıyan ve kullanıcıya sesli geri bildirim ile yön rehberliği sunan erişilebilir bir mobil uygulama tasarlanmış ve geliştirilmiştir.

Geliştirilen sistem, daha önceki tasarım çalışmasında ortaya konulan asgari uygulanabilir ürün (Minimum Viable Product) iskeletinin üzerine inşa edilmiş; nesne tanıma yeteneği COCO veri kümesi üzerinde eğitilmiş bir TensorFlow Lite SSD MobileNet modeli ile zenginleştirilmiş, yön bulma yeteneği ise Açık Sokak Haritası (OpenStreetMap) tabanlı OSRM ve Nominatim servisleriyle bütünleştirilmiştir. Bu sayede uygulama yalnızca çevresindeki engelleri sözel olarak adlandırmakla kalmamakta; aynı zamanda kullanıcıyı seçtiği bir hedefe doğru adım adım yönlendirmektedir.

Tasarım sürecinde teknik doğruluk kadar erişilebilirlik, kullanılabilirlik ve sürdürülebilirlik ilkeleri de ön planda tutulmuştur. Görme engelli bireylerin mobil cihazlarla etkileşim biçimleri dikkate alınarak, ekran okuyucularla uyumlu, sade ve kullanıcıyı yormayan bir arayüz benimsenmiştir. Bu yönüyle çalışma, yalnızca bir yazılım geliştirme süreci değil; insan odaklı mühendislik yaklaşımının somut bir uygulaması olarak ele alınmıştır.

Bu çalışma süresince, lisans eğitimi boyunca edinilen teorik bilgi ve teknik beceriler gerçek bir problem üzerinde uygulanmış; yazılım geliştirme süreci sistematik ve mühendislik temelli bir yaklaşımla yürütülmüştür. Çalışmanın hem akademik alanda hem de erişilebilirlik teknolojileri kapsamında anlamlı bir katkı sağlaması amaçlanmıştır.

Bu çalışmanın gerçekleştirilmesinde bilgi, deneyim ve yönlendirmeleriyle katkı sağlayan değerli danışmanımız Dr. Öğr. Üyesi Hüseyin ESKİ'ye teşekkür eder; çalışmanın ilham kaynağını oluşturan tüm görme engelli bireylere saygılarımızı sunarız.

İÇİNDEKİLER

ÖNSÖZ.....	iii
İÇİNDEKİLER.....	iv
SİMGELER VE KISALTMALAR LİSTESİ.....	vi
ŞEKİLLER LİSTESİ.....	vii
TABLolar LİSTESİ.....	viii
ÖZET.....	ix
BÖLÜM 1. GİRİŞ.....	1
1.1. Görme Engelli Bireylerin Günlük Yaşamda Karşılaştığı Problemler.....	2
1.2. Mevcut Teknolojik Çözümler ve Sınırlılıkları.....	3
1.3. Çalışmanın Amacı ve Kapsamı.....	4
1.4. Çalışmanın Akademik ve Toplumsal Katkısı.....	5
1.5. Tezin Organizasyonu.....	5
BÖLÜM 2. SİSTEMATİK YAKLAŞIM.....	6
2.1. Problem Formülasyonu.....	6
2.2. Donanım Mimarisi.....	7
2.3. Yazılım Mimarisi.....	8
2.3.1. Algılama katmanı.....	8
2.3.2. İşleme ve karar katmanı.....	9
2.3.3. Sunum ve erişilebilirlik katmanı.....	10
2.3.4. Navigasyon katmanı.....	11
2.3.5. Kullanılan teknolojiler ve geliştirme ortamı.....	12
2.4. Hibrit Çevrim İçi ve Çevrim Dışı Çalışma Yapısı.....	14
2.5. Kullanım Senaryoları ve İş Akış Diyagramları.....	14
2.6. Fonksiyonel Gereksinimler.....	16
2.7. Fonksiyonel Olmayan Gereksinimler.....	17
2.8. Algoritmik Yaklaşım ve Karar Mantığı.....	18
2.9. Modül Tasarımı ve Bileşen Sorumlulukları.....	20
2.10. Sistem Mimarisi ve Veri Akış Diyagramı.....	21
2.11. UML Diyagramları.....	22
2.12. Performans Hedefleri ve Kabul Kriterleri.....	23
BÖLÜM 3. DENEY DÜZENİĞİ VE SANAL LABORATUVAR.....	24
3.1. Deney Düzeniği.....	24

3.2. Deney Senaryoları.....	25
3.2.1. Çevrim dışı sokak modu senaryosu.....	25
3.2.2. Çevrim dışı iç mekan modu senaryosu.....	26
3.2.3. Çevrim içi navigasyon senaryosu.....	27
3.3. Sanal Laboratuvar Ortamı.....	28
3.4. Test Aşaması ve Değerlendirme Kriterleri.....	29
3.5. Performans Ölçümleri.....	30
3.6. Deneysel Bulguların Tartışılması.....	31
3.7. Geçerlilik Tehditleri ve Sınırlılıklar.....	32
3.8. İş Paketleri ve Zaman Planı.....	33
BÖLÜM 4. VERİ GÜVENLİĞİ DEĞERLENDİRMESİ.....	34
4.1. Toplanan Veri Türleri ve Güvenlik Düzeyleri.....	34
4.2. Veri İşleme Süreci ve Gizlilik Yaklaşımı.....	35
4.3. Veri İletimi ve Ağ Güvenliği.....	35
4.4. Mobil Uygulamalara Özgü Güvenlik Riskleri.....	36
4.5. Hukuki ve Etik Değerlendirme.....	36
4.6. Tehdit Modeli ve Saldırı Yüzeyi Analizi.....	37
4.7. İzin Yönetimi ve Asgari Yetki İlkesi.....	38
4.8. KVKK ve GDPR Perspektifi.....	38
BÖLÜM 5. SONUÇLAR VE ÖNERİLER.....	39
5.1. Çalışmadan Elde Edilen Sonuçlar.....	39
5.2. Akademik Katkıları.....	40
5.3. Sınırlılıklar.....	40
5.4. Gelecek Çalışmalar İçin Öneriler.....	41
5.5. MVP Çıktılarının Değerlendirilmesi.....	42
5.6. Literatürle Kıyas ve Konumlandırma.....	42
5.7. Uygulanabilirlik ve Yaygın Etki.....	43
KAYNAKLAR.....	44
EK A. EKLER.....	46
ÖZGEÇMİŞ.....	48
BSM 498 DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI.....	49

SİMGELER VE KISALTMALAR LİSTESİ

API	: Application Programming Interface (Uygulama Programlama Arayüzü)
APK	: Android Package Kit
CNN	: Convolutional Neural Network (Evrişimsel Sinir Ağı)
COCO	: Common Objects in Context (Yaygın Bağlamsal Nesneler veri kümesi)
CPU	: Central Processing Unit (Merkezi İşlem Birimi)
EAS	: Expo Application Services
FPS	: Frames Per Second (Saniyedeki Kare Sayısı)
GDPR	: General Data Protection Regulation
GPS	: Global Positioning System
GPU	: Graphics Processing Unit
HTTPS	: HyperText Transfer Protocol Secure
KVKK	: Kişisel Verilerin Korunması Kanunu
ML	: Machine Learning (Makine Öğrenmesi)
MVP	: Minimum Viable Product (Asgari Uygulanabilir Ürün)
OSM	: OpenStreetMap
OSRM	: Open Source Routing Machine
RAM	: Random Access Memory
REST	: Representational State Transfer
SDK	: Software Development Kit
SSD	: Single Shot MultiBox Detector
TFLite	: TensorFlow Lite
TLS	: Transport Layer Security
TTS	: Text-to-Speech (Metinden Konuşmaya)
UI	: User Interface (Kullanıcı Arayüzü)
UX	: User Experience (Kullanıcı Deneyimi)
WCAG	: Web Content Accessibility Guidelines
WHO	: World Health Organization (Dünya Sağlık Örgütü)
YUV	: Luma Chrominance renk uzayı

ŞEKİLLER LİSTESİ

Şekil 2.1. Sistem mimarisi katmanlı diyagramı.....	21
Şekil 2.2. Veri akış diyagramı.....	21
Şekil 2.3. Use Case diyagramı.....	22
Şekil 2.4. Algılama pipeline sequence diyagramı.....	22
Şekil 2.5. Navigasyon pipeline sequence diyagramı.....	23
Şekil 2.6. Mod geçişleri durum makinesi.....	23
Şekil 2.7. Bileşen (component) diyagramı.....	24
Şekil 3.1. Test akış diyagramı.....	29
Şekil 3.2. Mod bazlı ortalama FPS karşılaştırması.....	30
Şekil 3.3. VisionAssist ana ekran tasarımı.....	26
Şekil 4.1. Tehdit modeli ve veri akışı.....	37
Şekil 5.1. MVP hedef ve gerçekleşen tamamlama oranları.....	42

TABLÖLAR LİSTESİ

Tablo 2.1. Geliştirme ortamı ve kullanılan teknolojiler.....	13
Tablo 2.2. Fonksiyonel gereksinimler.....	16
Tablo 2.3. Erişilebilirlik kabul kriterleri (WCAG karşılığı).....	18
Tablo 3.1. Ortam bazlı algılama performansı.....	30
Tablo 3.2. Ekip sorumlulukları.....	33
Tablo 4.1. İzin, veri ve amaç eşlemesi.....	38
Tablo 5.1. Görme engelli mobil sistemler karşılaştırması.....	42
Tablo 5.2. Karşılaşılan sorunlar ve çözümleri.....	41

ÖZET

Anahtar kelimeler: Görme engelliler, erişilebilirlik, gerçek zamanlı nesne tanıma, sesli yön tarifi, mobil rehber sistem, derin öğrenme, TensorFlow Lite

Görme engelli ve az gören bireylerin günlük yaşamda güvenli ve bağımsız hareket edebilmesi, erişilebilirlik ve insan odaklı mühendislik alanında öne çıkan kritik bir problemdir. Açık alanlardaki düzensiz kaldırımlar, kontrolsüz park edilen araçlar, geçici yol çalışmaları ve dinamik çevresel engeller, bu bireyler için ciddi güvenlik riskleri oluşturmakta; mevcut harita ve yön bulma uygulamaları çevredeki anlık nesneleri algılayamadığı için yetersiz kalmaktadır.

Bu bitirme çalışmasında, görme engelli ve az gören bireylerin çevresel farkındalığını artırmayı ve hedef yönlendirme ihtiyacını karşılamayı amaçlayan, gerçek zamanlı çalışan bir mobil rehber uygulamasının tasarımı ve geliştirilmesi ele alınmıştır. Geliştirilen VisionAssist uygulaması, mobil cihazın arka kamerasından alınan görüntüleri TensorFlow Lite üzerinde çalışan bir SSD MobileNet nesne tanıma modeline ileterek tespit edilen nesnelerin sınıfını, yönünü ve mesafesini Türkçe sözel ifadelerle dönüştürmekte; eş zamanlı olarak OpenStreetMap tabanlı OSRM ve Nominatim servisleri üzerinden kullanıcıyı seçtiği bir hedefe adım adım yönlendirmektedir.

Sistem React Native ve Expo altyapısı üzerinde, Vision Camera ile yüksek hızlı kare yakalama, Worklets çekirdeği ile JavaScript dışı senkron çıkarım, Expo Speech ile metinden konuşmaya ve Expo Location ile canlı konum izleme bileşenleri kullanılarak modüler ve katmanlı bir mimariye geliştirilmiştir. Çevrim dışı senaryoda nesne tanıma ve sesli geri bildirim cihaz üzerinde çalışırken; çevrim içi senaryoda harita ve yol tarifi servisleri devreye girerek hibrit bir çalışma yapısı sağlanmıştır.

Erişilebilirlik gereklilikleri W3C WCAG 2.1 standardı ve mobil platformların ekran okuyucu çerçeveleriyle (TalkBack ve VoiceOver) uyumlu olacak şekilde karşılanmış; tüm etkileşim öğeleri için 56 piksel asgari dokunma alanı, yüksek kontrast renk paleti ve eş zamanlı sesli geri bildirim sağlanmıştır. Çalışma kapsamında geliştirilen uygulama, gerçek dünya koşullarında üç farklı çalışma modu (sokak, iç mekân, navigasyon) altında test edilmiş; saniye başına ortalama 16 ile 22 arasında kare işleyebildiği, kritik engel uyarılarının ortalama bir saniyenin altında üretildiği ve hedefe yönlendirmenin yaya rotalarında doğru biçimde gerçekleştiği gözlenmiştir.

Elde edilen sonuçlar, mobil cihazlar üzerinde çalışan erişilebilir yazılım çözümlerinin, görme engelli bireylerin günlük yaşam güvenliğini ve bağımsızlığını anlamlı ölçüde artırma potansiyeline sahip olduğunu göstermektedir. İlerleyen çalışmalarda; görme engelliler için özel olarak etiketlenmiş veri kümeleri ile yeniden eğitim, derinlik tahmini için cihaz üstü özel modeller, sesli komutla etkileşim ve çoklu dil desteği gibi yönlerde sistemin geliştirilmesi öngörülmektedir.

BÖLÜM 1. GİRİŞ

Görme engelli ve az gören bireylerin günlük yaşamda bağımsız hareket edebilmesi, modern toplumlarda erişilebilirlik ve kapsayıcılık anlayışının temel göstergelerinden biri olarak kabul edilmektedir. Dünya Sağlık Örgütü tarafından 2019 yılında yayımlanan Görme Üzerine Dünya Raporu, dünya genelinde en az 2,2 milyar insanın görme bozukluğu ile yaşadığını ve bu sayının yaşlanan nüfus ve dijitalleşmenin etkisi ile artmaya devam edeceğini ortaya koymaktadır [1]. Görme duyusunun kısmen ya da tamamen kaybı; bireylerin çevresel nesneleri algılama, yön bulma ve güvenli hareket etme yetilerini doğrudan etkilemekte, özellikle dış mekânlarda bağımsız hareketi önemli ölçüde sınırlandırmaktadır.

Şehir yaşamında karşılaşılan düzensiz fiziksel koşullar; kaldırımlara taşan dükkân eşyaları, kontrolsüz park edilen araçlar, geçici yol çalışmaları, kapanmamış rögarlar ve sabit olmayan çevresel engeller, görme engelli bireyler için ciddi güvenlik riskleri oluşturmaktadır. Yapılan saha çalışmaları, bu tür çevresel faktörlerin görme engelli bireylerde düşme, çarpma ve yaralanma riskini anlamlı ölçüde artırdığını ortaya koymaktadır [2]. Bu nedenle, görme engelli bireylerin çevrelerini algılamalarına yardımcı olacak teknolojik çözümler, yalnızca bireysel yaşam kalitesini artırmakla kalmamakta; aynı zamanda toplumsal katılım ve güvenli yaşam açısından da kritik bir önem taşımaktadır.

Bu çalışmada, görme engelli ve az gören bireylerin çevresel farkındalığını artırmayı amaçlayan, mobil cihazlar üzerinde çalışan ve gerçek zamanlı geri bildirim sağlayan bir rehber uygulamanın tasarımı ve geliştirilmesi ele alınmaktadır. Çalışma, erişilebilir yazılım geliştirme ilkeleri doğrultusunda şekillendirilmiş olup; kullanıcıların ek bir donanıma ihtiyaç duymadan yalnızca akıllı telefonları aracılığıyla çevrelerini algılayabilmelerini ve istedikleri bir hedefe sesli olarak yönlendirilebilmelerini hedeflemektedir. Mobil erişilebilirlik alanında yapılan önceki çalışmalar, kamera ve sesli geri bildirim temelli çözümlerin görme engelli bireyler için umut vadeden yaklaşımlar sunduğunu göstermektedir [2], [3].

Geliştirilen sistem, daha önceki tasarım çalışmasında ortaya konulan asgari uygulanabilir ürün iskeletinin üzerine inşa edilmiştir. Önceki sürüm yalnızca yön ve mesafeye dayalı genel uyarılar üretebilirken; bu çalışmada gerçekleştirilen iyileştirmelerle uygulama çevredeki nesneleri sınıf adıyla (sandalye, masa, kişi,

otomobil, bisiklet vb.) tanıyabilen ve kullanıcıyı OpenStreetMap tabanlı bir altyapı üzerinden adım adım yönlendirebilen bir sisteme dönüştürülmüştür. Bu dönüşüm, kullanıcı deneyimini önemli ölçüde derinleştirmekte ve uygulamayı bir “engel uyarıcısı” olmaktan çıkarıp gerçek anlamda bir “mobil rehber sisteme” yaklaştırmaktadır.

1.1. Görme Engelli Bireylerin Günlük Yaşamda Karşılaştığı Problemler

Görme engelli bireyler, özellikle açık alanlarda hareket ederken çok sayıda fiziksel engelle karşılaşmaktadır. Kaldırımlara taşan dükkân eşyaları, düzensiz park edilen araçlar, yol çalışmaları, merdiven girişleri, bariyerler ve sokak tabelaları gibi unsurlar, bu bireyler için ciddi tehlikeler oluşturmaktadır. Geleneksel beyaz baston, yalnızca fiziksel temas sağlanan engelleri algılayabilmekte; temas öncesinde tehlike oluşturan nesneler hakkında yeterli bilgi sunamamaktadır. Literatürde, baston kullanımının tek başına çevresel farkındalık için yeterli olmadığı ve destekleyici teknolojilere ihtiyaç duyulduğu vurgulanmaktadır [2], [4].

Mevcut sesli navigasyon ve harita tabanlı uygulamalar ise ağırlıklı olarak konum ve yön bilgisi sağlamaya odaklanmakta; çevredeki anlık engelleri ve dinamik nesneleri algılayamamaktadır. Bu durum, görme engelli bireylerin yalnızca “nereye gideceklerini” bilmelerini sağlamakta, ancak “önlerinde ne olduğu” bilgisini sunamamaktadır. Dolayısıyla güvenli hareket; yalnızca navigasyon desteğiyle değil, çevresel farkındalık sağlayan gerçek zamanlı sistemlerle mümkün olabilmektedir [3]. Bu bilgi açığı, mobil cihazların kamera ve işlem gücü kapasitelerinin son yıllarda hızla artmasıyla birlikte, cihaz üstü yapay zekâ tabanlı çözümlerle kapatılmaya başlanmıştır [5], [6].

Ayrıca iç mekânlarda da problemin farklı bir boyutu bulunmaktadır. Bilinmeyen bir kafe, hastane veya ofis ortamında masaların, sandalyelerin, koltukların, kapı tutamaklarının ve diğer mobilyaların konumu; görme engelli bir birey için önceden öngörülebilir bilgi olmadığında çarpma ve düşme riskini artırmaktadır. İç mekân koşulları aydınlatma değişkenliği nedeniyle algoritmik olarak da farklı zorluklar barındırır; bu nedenle uygulamanın sokak ve iç mekân için ayrı çalışma profillerine sahip olması zorunlu görülmüştür.

1.2. Mevcut Teknolojik Çözümler ve Sınırlılıkları

Son yıllarda görme engelli bireylere yönelik çeşitli teknolojik çözümler geliştirilmiştir. Mobil platformlar üzerinde Microsoft Seeing AI, Google Lookout,

Envision AI ve OrCam MyEye gibi uygulamalar; nesne tanıma, metin okuma, yüz tanıma ve sahne betimleme gibi temel işlevleri sunmaktadır [3], [7]. Bunların yanı sıra Be My Eyes ve gönüllü tabanlı çağrı uygulamaları, görme engelli bireylere uzaktaki gönüllüler aracılığıyla canlı yardım sağlamaktadır. Ancak bu çözümlerin tamamı, belirli sınırlılıklar barındırmaktadır:

- Çoğu uygulama, nesneleri yalnızca tanımlamakta; nesnenin kullanıcıya göre yönü, mesafesi ve oluşturduğu risk hakkında yeterli bilgi sunmamaktadır.
- Çevrim içi modele bağımlı sistemler, internet bağlantısının olmadığı durumlarda işlevselliğini kaybetmektedir. Bu durum, özellikle yer altı geçitleri, kapalı otoparklar ve kırsal alanlarda kullanıcıyı tehlikeye atabilmektedir.
- OrCam gibi donanım tabanlı çözümler, yüksek maliyet ve sınırlı taşınabilirlik nedeniyle her kullanıcı için erişilebilir değildir.
- Birçok uygulamanın kullanıcı arayüzü, ekran okuyucu uyumluluğu açısından eksik tasarlanmıştır. Erişilebilirlik etiketleri eksik, dokunma alanları küçük ve sesli geri bildirim mekanizmaları zayıf olabilmektedir [8].
- Görme engelli bireylere yönelik özel olarak etiketlenmiş veri kümesi sayısının sınırlı olması, akademik çalışmaların büyük ölçüde genel amaçlı veri kümeleri (örneğin COCO) üzerinden yürütülmesine yol açmakta; bu durum sokakta sıkça karşılaşılan ve kritik tehlike içeren bazı nesnelerin (örneğin yer kazısı, çukur, asılı dal) tanınamaması sorununu doğurmaktadır [9].

Bu sınırlılıklar, görme engelli bireyler için gerçek zamanlı, erişilebilir, çevrim dışı çalışabilen ve aynı zamanda hedef yönlendirme sunan bütünleşik bir sistem ihtiyacının hâlen güncel olduğunu göstermektedir. Bu çalışma, söz konusu boşluğu doldurmaya yönelik mütevazı ancak somut bir teknik katkı olarak değerlendirilebilir.

1.3. Çalışmanın Amacı ve Kapsamı

Bu bitirme çalışmasının temel amacı, görme engelli ve az gören bireylerin çevrelerini daha güvenli ve bağımsız bir şekilde algılayabilmelerini ve istedikleri hedeflere sesli rehberlik eşliğinde ulaşabilmelerini sağlamak üzere, mobil cihazlar üzerinde çalışan erişilebilir bir rehber uygulama geliştirmektir. Geliştirilen VisionAssist sistemi; mobil cihazın arka kamerası aracılığıyla çevreden görüntü almakta, görüntüleri yapay zekâ tabanlı bir nesne tanıma modeline ileterek tespit edilen nesneleri Türkçe sözel ifadelerle kullanıcıya bildirmekte ve eş zamanlı olarak konum servislerini kullanarak kullanıcıyı seçtiği bir hedefe yönlendirmektedir.

Çalışma kapsamında aşağıdaki teknik hedefler tanımlanmıştır:

- Platform bağımsız bir mobil uygulama geliştirilmesi (Android öncelikli, mimari iOS uyumlu).
- Erişilebilirlik öncelikli bir kullanıcı arayüzünün tasarlanması; ekran okuyucu, yüksek kontrast ve geniş dokunma alanı kullanımı.
- Çevrim içi ve çevrim dışı kullanım senaryolarının desteklendiği hibrit bir mimari oluşturulması.
- Cihaz üzerinde çalışan TensorFlow Lite tabanlı bir nesne tanıma modelinin entegrasyonu ve gerçek zamanlı kare işleme.
- OpenStreetMap tabanlı OSRM ve Nominatim servislerinin kullanılarak yaya yol tarifi ve adım adım sesli rehberlik sağlanması.
- Sesli geri bildirim mekanizmasının önceliklendirme, yumuşatma (smoothing) ve hata toleransı içerecek şekilde tasarlanması.

Çalışma, aynı zamanda bir asgari uygulanabilir ürün geliştirilmesini hedeflemekte olup, ileri aşamalarda yapılabilecek özel veri kümesi entegrasyonları, derinlik tahmini ve sesli komut etkileşimi gibi geliştirmeler için ölçeklenebilir bir altyapı sunmaktadır [10]. Sistem, sokak, iç mekan ve navigasyon olmak üzere üç farklı çalışma modunu destekleyecek şekilde tasarlanmıştır; bu sayede dış çevre koşullarının değişen niteliğine göre algılama ve seslendirme parametrelerinin uyarlanması mümkün hale gelmiştir.

1.4. Çalışmanın Akademik ve Toplumsal Katkısı

Bu çalışma, bilgisayar mühendisliği ve erişilebilir yazılım geliştirme alanlarında hem akademik hem de toplumsal katkı sunmayı amaçlamaktadır. Akademik açıdan, mobil platformlar üzerinde çalışan erişilebilir sistemlerin tasarımına ilişkin bütünlük bir uygulama örneği sunulmakta; nesne tanıma, sesli geri bildirim ve harita tabanlı yönlendirmenin tek bir mimari altında birleştirilmesinin mühendislik gerekçeleri ortaya konmaktadır. Çalışma, derin öğrenme tabanlı algılama bileşenleri ile klasik harita servislerinin hibrit kullanımına ilişkin somut bir referans modeli oluşturmaktadır.

Toplumsal açıdan ise çalışma, görme engelli bireylerin günlük yaşam güvenliğini ve bağımsızlığını artırmaya yönelik bir teknolojik çözüm önermektedir. İnsan odaklı teknoloji anlayışını temel alan bu yaklaşım, yapay zekâ ile mobil yazılım teknolojilerinin sosyal fayda üretme potansiyelini ortaya koymakta ve erişilebilirlik

teknolojilerinin yaygınlaşmasına katkıda bulunmayı hedeflemektedir. Geliştirilen sistemin açık kaynak ekosistemi (React Native, TensorFlow Lite, OpenStreetMap, OSRM) üzerine inşa edilmiş olması, gelecekte aynı problem üzerinde çalışacak araştırmacılar için referans olarak değerlendirilmesini de mümkün kılmaktadır.

1.5. Tezin Organizasyonu

Tezin geri kalanı şu şekilde organize edilmiştir. İkinci bölümde sistemin tasarım ilkeleri, donanım ve yazılım mimarisi, kullanılan teknolojiler, fonksiyonel ve fonksiyonel olmayan gereksinimler ile algoritmik karar mantığı ayrıntılı biçimde sunulmaktadır. Üçüncü bölümde geliştirilen sistemin gerçek cihaz üzerinde test edildiği deney düzeneği, senaryolar, performans ölçüm sonuçları ve değerlendirme kriterleri ele alınmaktadır. Dördüncü bölüm, sistemin işlediği veri türlerini, izin yönetimi politikasını, tehdit modelini ve KVKK ile GDPR perspektifinden hukuki değerlendirmeyi kapsamaktadır. Beşinci bölümde elde edilen sonuçlar, akademik katkılar, sınırlılıklar ve gelecek çalışmalar için öneriler tartışılmaktadır. Tezin sonunda kullanılan kaynaklar, ekler ve özgeçmişler yer almaktadır.

BÖLÜM 2. SİSTEMATİK YAKLAŞIM

Bu bölümde, görme engelli ve az gören bireyler için geliştirilen VisionAssist mobil rehber uygulamasının sistematik tasarım yaklaşımı, mimari bileşenleri, veri işleme süreçleri ve tasarım kararlarının mühendislik gerekçeleri ayrıntılı olarak sunulmaktadır. Sistematik yaklaşım belirlenirken; erişilebilirlik, gerçek zamanlı çalışma gereksinimi, mobil cihazların donanımsal sınırlamaları, kullanıcı güvenliği, çevrim içi/çevrim dışı esneklik ve geliştirilebilirlik temel kriterler olarak ele alınmıştır. Literatürde görme engelli bireyler için geliştirilen benzer sistemler incelenmiş [3], [7]; mevcut çalışmaların sınırlılıkları göz önünde bulundurularak bütünlüklük bir mimari tasarlanmıştır.

2.1. Problem Formülasyonu

Bu çalışmada ele alınan temel problem; görme engelli bireylerin fiziksel çevrelerini görsel girdiler aracılığıyla algılayamaması, dolayısıyla güvenli hareket edememesi ve kendilerine ait olmayan ortamlarda yön bulamamasıdır. Problem, mühendislik bakış açısıyla bir algılama – yorumlama – iletişim – yönlendirme zinciri olarak modellenmiştir. Sistem formülasyonu aşağıdaki bileşenlerden oluşmaktadır:

- Girdi: Mobil cihazın arka kamerasından alınan anlık görüntü kareleri, eş zamanlı GPS konum bilgisi ve kullanıcı tercih ayarları (mod, hassasiyet, dil, konuşma hızı).
 - İşlem: Görüntü karelerinin ön işlenmesi, derin öğrenme tabanlı sınıflandırma, tespit edilen nesnelerin yönüne ve mesafesine göre sınıflandırılması, risk değerlendirmesi, yol tarifi hesaplama ve adım eşleme.
 - Çıktı: Kullanıcıya iletilen Türkçe (veya İngilizce) sözel uyarılar ve yön talimatları, görsel arayüzde gösterge ve titreşim geri bildirimi.
 - Geri Besleme: Kullanıcının cihaz konumunu sürekli izleyerek bir sonraki yönlendirme adımının tetiklenmesi ve seslendirilen mesajın tekrar talep edilebilmesi.
- Bu formülasyon, literatürde görme engelliler için önerilen bağlam farkındalığı (context-aware systems) yaklaşımıyla uyumludur [3]. Amaç, kullanıcıya çevredeki tüm bilgiyi değil, hareket güvenliğini doğrudan etkileyen ve hedefe ilerlemeyi destekleyen bilgiyi aktarmaktır. Bu sayede bilişsel yük azaltılmakta ve kullanıcı deneyimi iyileştirilmektedir.

2.2. Donanım Mimarisi

Geliştirilen sistem, ek bir donanım veya özel sensör gerektirmeden, yalnızca akıllı telefon donanımı üzerinde çalışacak şekilde tasarlanmıştır. Bu tercih, sistemin erişilebilirliğini önemli ölçüde artırmakta ve maliyet açısından sürdürülebilir bir çözüm sunmaktadır. Literatürde yapılan çalışmalar, ek donanım gerektiren sistemlerin kullanıcılar tarafından benimsenme oranının düşük olduğunu ve uzun vadede sürdürülebilir olmadığını göstermektedir [2], [4]. Sistemde kullanılan temel donanım bileşenleri şunlardır:

- Arka kamera: Çevresel görüntülerin gerçek zamanlı yakalanması için kullanılır. Sistem, en az 8 megapiksel çözünürlüğe ve 30 FPS kare hızına sahip bir kamera ile etkin biçimde çalışmaktadır.
- GPS modülü: Çevrim içi navigasyon senaryolarında konum bilgisi sağlamak amacıyla kullanılır. Yüksek doğruluk modu (high accuracy) seçilmiş; her 2 saniyede bir veya 5 metrelik hareketlerde konum güncellemesi alınmaktadır.
- Dahili hoparlör: Sesli geri bildirim için cihazın varsayılan ses çıkışı kullanılmaktadır. Bluetooth kulaklık desteği, işletim sistemi düzeyinde devralınmaktadır.
- Titreşim motoru: Buton etkileşimlerinde geri bildirim sağlamak için isteğe bağlı dokunsal uyarı üretir. Görme engelli kullanıcının dokunduğu öğeyi doğrulamasına yardımcı olur.
- İşlemci ve grafik birimi: TFLite modeli, yapay zekâ çıkarımını cihaz üstünde gerçekleştirmek için işlemci ya da varsa grafik işlemcisi (delegate) kullanmaktadır. Test sırasında kullanılan Snapdragon 7 sınıfı işlemcili cihazlarda CPU üzerinde 16-22 FPS bandında çalışma elde edilmiştir.

Donanım mimarisi, farklı marka ve model cihazlar arasında uyumluluğu koruyacak şekilde esnek olarak tasarlanmıştır. Kamera çözünürlüğü, kare hızı ve işlemci performansı gibi değişkenler yazılım katmanında dinamik olarak yönetilmekte; cihazın kapasitesine göre algılama aralığı ve karelerin çözünürlüğü ayarlanmaktadır. Bu uyarlama; CameraService bileşeni içinde, ardışık kareler arasındaki bekleme süresinin (capture interval) ve kare çözünürlüğünün adaptif olarak ayarlanması yoluyla gerçekleştirilmektedir.

2.3. Yazılım Mimarisi

Yazılım mimarisi, modüler ve katmanlı bir yapı üzerine kurulmuştur. Bu yaklaşım, yazılım mühendisliğinde sürdürülebilirlik ve genişletilebilirlik açısından yaygın olarak tercih edilen bir desen olup; her katmanın yalnızca üst ve alt katmanları ile etkileşmesini sağlayarak bağımlılıkları en aza indirmektedir. Sistem mimarisi dört ana katmanda ele alınmıştır.

2.3.1. Algılama katmanı

Algılama katmanı, mobil cihaz kamerasından elde edilen görüntülerin alındığı, yapay zekâ modelinin çalıştığı ve konumun izlendiği katmandır. Görüntü kareleri react-native-vision-camera kütüphanesi üzerinden yakalanmaktadır. Vision Camera'nın frame processor mekanizması, JavaScript tek iş parçacığının tıkanmasını önlemek amacıyla react-native-worklets-core üzerinde, JavaScript bağlamından bağımsız bir thread'te çalışmaktadır. Bu sayede her saniye 30 kareye kadar yakalama mümkün olmakta; her bir karenin model çıkarımı için işlenmesi sırasında arayüz akıcılığı korunmaktadır.

Yakalanan ham kare, react-native-fast-tf-lite kütüphanesi tarafından sunulan TFLite çalıştırıcısı için 384×384 piksel RGB ve uint8 formatına vision-camera-resize-plugin aracılığıyla yeniden boyutlandırılır. Bu boyut, kullanılan SSD MobileNet v1 modelinin beklediği giriş tensörüne uygundur. Konum bilgisi ise expo-location modülü üzerinden, yüksek doğruluk modunda, 5 metrelik hareketlerde veya 2 saniye aralıklarla güncellenecek biçimde izlenmektedir.

2.3.2. İşleme ve karar katmanı

Bu katman, sistemin en kritik bölümünü oluşturmaktadır. Algılanan görsel veriler bu aşamada analiz edilmekte, tespit edilen nesneler kullanıcı için anlamlı bir öncelik sırasına dönüştürülmekte ve seslendirilecek mesajların içeriği belirlenmektedir. Literatürde önerilen karar destek mekanizmalarına benzer şekilde, gereksiz ve tekrarlayan bildirimler filtrelenmektedir [3].

Karar katmanı içinde aşağıdaki alt bileşenler yer almaktadır:

- **ObstacleDetector:** Algılama pipeline'ının orkestratörüdür. FrameProcessor'dan gelen tespitleri DirectionAnalyzer, DistanceEstimator ve RiskEvaluator'a yönlendirir.
- **DirectionAnalyzer:** Tespit edilen nesnenin sınırlayıcı kutusunun ekrandaki x eksenini kullanarak nesnenin sol, orta veya sağ yönde olduğunu belirler.

- DistanceEstimator: Sınırlayıcı kutunun alanını ve mod parametrelerini kullanarak yakın, orta veya uzak mesafe sınıflandırması üretir; bu sınıflandırma kullanıcının hareketinin aciliyetini belirler.
- RiskEvaluator: Yön ve mesafe verisini birleştirerek HIGH, MEDIUM, LOW ve NONE risk seviyelerini hesaplar; aynı anda birden fazla nesne tespit edildiğinde en yüksek riskli olanı önceliklendirir.
- NavigationContext: Aktif rota varsa, kullanıcının mevcut konumunu rota adımları ile karşılaştırarak bir sonraki adımı belirler ve sesli olarak bildirilmesini tetikler.

Bu yaklaşım, güvenlik açısından kritik durumların ön plana çıkarılmasını sağlamakta, aynı zamanda navigasyon yönlendirmelerinin engel uyarıları ile çakışmadığı bir önceliklendirme şeması sunmaktadır.

2.3.3. Sunum ve erişilebilirlik katmanı

Sunum katmanı, sistem çıktılarının kullanıcıya iletildiği bölümdür. Görsel arayüz React Native bileşenleri ile geliştirilmiş olup; tüm etkileşim öğeleri en az 56 piksel dokunma alanına sahiptir, yüksek kontrast (siyah-beyaz ağırlıklı) renk paleti kullanılmaktadır. Tüm butonlar accessibilityLabel, accessibilityHint ve accessibilityRole özellikleri ile etiketlenmiş; bu sayede TalkBack ve VoiceOver gibi ekran okuyuculardan da erişilebilir hale getirilmiştir.

Sözel geri bildirim için expo-speech kütüphanesi tercih edilmiştir. Önceki tasarım çalışmasında çıktının doğrudan ekran okuyucuya bırakılması düşünülmüş; ancak gerçek cihaz testlerinde bu yaklaşımın gecikmeli olduğu, bazı ekran okuyucu sürümlerinin kısa metinleri atladığı ve dil ayarına bağlı olarak sesin yanlış aksanla okunabildiği gözlenmiştir. Bu nedenle uygulama, kendi SpeechService bileşeni üzerinden TTS motorunu doğrudan çağırmakta; aynı mesajın kısa süre içinde tekrarlanmasını engellemek için 2 saniyelik bir cooldown uygulamakta ve ardışık beş hatadan sonra TTS motorunu otomatik olarak devre dışı bırakarak kullanıcıyı uyarmaktadır.

2.3.4. Navigasyon katmanı

Navigasyon katmanı, kullanıcının seçtiği bir hedefe doğru sesli yönlendirme sağlayan modüldür. Bu katman, üç ana bileşen üzerine inşa edilmiştir: yer arama (geocoding), yol tarifi hesaplama (routing) ve canlı konum takibi. Yer arama için OpenStreetMap topluluğunun ücretsiz Nominatim servisi, yol tarifi için aynı ekosistemin parçası olan OSRM (Open Source Routing Machine) servisinin halka

açık demo örneği kullanılmaktadır. Bu tercihin temel gerekçesi; Google Maps Directions API gibi servislerin proje ölçeğinde maliyetli olması ve API anahtarı yönetimini gerektirmesidir [11], [12].

Hedef belirleme süreci, görme engelli kullanıcılar için özel olarak tasarlanmıştır. Kullanıcı, navigasyon modunu seçtiğinde otomatik olarak DestinationPickerScreen ekranına yönlendirilir. Bu ekranda; en yakın eczane, hastane, market, otobüs durağı, kafe ve park gibi kategoriler büyük dokunma alanlı kartlar olarak sunulmaktadır. Kullanıcı bir kategoriye dokunduğunda, NavigationService o kategoriye mevcut konuma yakın olacak şekilde Nominatim üzerinden arar ve en yakın sonucu seçerek OSRM'den yaya yol tarifi alır.

Hesaplanan rota, OSRM tarafından sağlanan adımlara (steps) ayrıştırılır. Her bir adım; bir manevra türü (turn, depart, arrive, roundabout vb.), bir yön değiştirme niteliği (left, right, straight, slight left, sharp right vb.) ve adımın bittiği coğrafi noktanın koordinatlarını içerir. NavigationService bu adımları Türkçe sesli komutlara çevirir; örneğin OSRM tarafından üretilen “turn left” manevrası “200 metre sonra sola dönün” cümlesine, “arrive” manevrası “Hedefinize ulaştınız” bildirimine dönüştürülür.

Canlı izleme aşamasında, expo-location'dan gelen her konum güncellemesi mevcut adımın bitiş noktasına olan mesafeyi yeniden hesaplar. Bitişe 60 metre kala kullanıcıya “Yaklaşık 60 metre sonra: ...” şeklinde bir hatırlatma yapılır; 25 metre kala otomatik olarak bir sonraki adıma geçilir. Hedefe varıldığında “Hedefinize ulaştınız” bildirimi yapılır ve rota durumu temizlenir.

2.3.5. Kullanılan teknolojiler ve geliştirme ortamı

Çalışmada platform bağımsız geliştirme, hızlı prototipleme ve erişilebilirlik uyumluluğu hedefleri doğrultusunda React Native ve Expo SDK 54 altyapısı tercih edilmiştir. Uygulama; kamera erişimi, izin yönetimi, gerçek zamanlı görüntü işleme akışı, ekran okuyucu uyumlu arayüz bileşenleri ve harita servisleri entegrasyonu dikkate alınarak modüler biçimde yapılandırılmıştır. Geliştirme sürecinde kullanılan başlıca yazılım bileşenleri ve sürümleri Tablo 2.1'de özetlenmiştir.

Tablo 2.1. Geliştirme Ortamı ve Kullanılan Teknolojiler

Kategori	Bileşen ve Sürüm	Amaç
Çatı (framework)	React Native 0.81.5, Expo SDK 54	Platform bağımsız mobil geliştirme
Programlama dili	TypeScript 5.9, JavaScript	Tip güvenli uygulama mantığı

	ES2022	
Kamera erişimi	react-native-vision-camera 4.7	Yüksek hızlı kare yakalama, frame processor
Yardımcı işlemcili çıkarım	react-native-fast-tflite 2.0	TFLite SSD MobileNet model çalıştırma
Worklet altyapısı	react-native-worklets-core 1.6	JS dışı senkron çıkarım
Görüntü ölçekleme	vision-camera-resize-plugin 3.2	384×384 RGB uint8 yeniden boyutlandırma (EfficientDet-Lite1)
Sesli geri bildirim	expo-speech 14.0	Türkçe ve İngilizce TTS
Konum servisi	expo-location 19.0	Canlı GPS konum takibi
Dosya/medya	expo-file-system 19, expo-asset 12	Asset ve geçici dosya yönetimi
Görsel ölçekleme	expo-image-manipulator 14	Asenkron resim işleme
Erişilebilirlik	React Native Accessibility API	TalkBack, VoiceOver uyumu
Bildirim/dokunma	expo-haptics 15	Buton geri bildirimi
Navigasyon (uygulama içi)	@react-navigation/native 7, native-stack 7, bottom-tabs 7	Sekme ve modal akış
Yer arama	Nominatim Public API	Geocoding
Yol tarifi	OSRM Public Demo (foot)	Yaya rotalama
Yapay zekâ modeli	EfficientDet-Lite1 (COCO 90 sınıf, 5.8 MB)	Cihaz üstü nesne tanıma, 384×384 giriş, mAP@50-95: 30.6
Yerel depolama	@react-native-async-storage/async-storage 2.2	Kullanıcı ayarlarının saklanması
Build sistemi	EAS Build (cloud)	APK/AAB üretimi
Test cihazı	Android 14, Snapdragon 7 sınıfı	Fonksiyonel doğrulama ve performans
Versiyon kontrol	Git, GitHub	Sürüm yönetimi ve izlenebilirlik

Geliştirme sürecinde Visual Studio Code editörü, Cursor yardımcılı geliştirme ortamı ve Android Studio profil araçları kullanılmıştır. Uygulamanın hata izleme ve performans gözlemlenmesinde Metro bundler tarafından sağlanan log akışı ile gerçek zamanlı debug işlemleri için react-native-debugger kullanılmıştır.

2.3.6. Donanım Yapılandırma ve Platform Gereksinimleri

VisionAssist'in Android platformunda çalıştırılabilmesi için gerekli yazılım yapılandırma gereksinimleri aşağıda özetlenmiştir. Bu gereksinimler; uygulamanın kullandığı native modüllerin (Vision Camera, Fast TFLite, expo-location) Android sisteminden talep ettiği API düzeyleri ve izinler temel alınarak belirlenmiştir.

Tablo 2.4. Donanım Yapılandırma ve Platform Gereksinimleri

Parametre	Değer	Gerekçe
Android minSdkVersion	24 (Android 7.0)	react-native-vision-camera V4

		minimum gereksinimi
Android targetSdkVersion	35 (Android 15)	Google Play yayın politikası zorunluluğu
Gerekli izin: Kamera	android.permission.CAMERA	Gerçek zamanlı kamera erişimi
Gerekli izin: Konum	ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION	GPS tabanlı navigasyon ve yaya konumu
Gerekli izin: Titreşim	android.permission.VIBRATE	Dokunsal geri bildirim
Minimum RAM	2 GB	TFLite modeli ve kamera buffer için
Minimum depolama (kurulum)	~80 MB	APK, model dosyası (~4.4 MB) ve JS bundle
Kamera çözünürlüğü	≥ 8 MP, ≥ 30 FPS	Güvenilir nesne tespiti için minimum
iOS (gelecek)	iOS 13+ (planlama aşamasında)	Expo SDK 54 ve Vision Camera v4 uyumu

Native modüllerin yapılandırması EAS Build cloud servisi aracılığıyla gerçekleştirilmektedir. Lokal geliştirme ortamında NDK 27 ve CMake 3.22 kurulumu gerekmektedir. Bu yapılandırma gereksinimleri app.json ve android/build.gradle dosyalarında sabitlenmiştir.

2.3.7. Veri Saklama Yaklaşımı ve Veritabanı Kararı

Uygulamada geleneksel bir ilişkisel veya NoSQL veritabanı kullanılmamaktadır. Bu tasarım kararı; uygulamanın sakladığı verinin yapısal açıdan basit (anahtar-değer çiftleri), hacimsel açıdan küçük ve kullanıcıya özgü olmasından kaynaklanmaktadır. Veritabanı kullanılmaması gizlilik avantajı da sağlamaktadır: kullanıcı verileri hiçbir uzak sunucuya gönderilmemekte, tamamen cihazda saklanmaktadır.

Kalıcı veri saklama ihtiyacı @react-native-async-storage/async-storage kütüphanesi ile karşılanmaktadır. Bu kütüphane; Android'de SQLite tabanlı bir anahtar-değer deposu üzerinde çalışmakta, her oturumda kullanıcı ayarlarını (mod, konuşma hızı, dil, titreşim, hassasiyet eşiği) ve son seçilen modu geri yüklemektedir. Kavramsal veri modeli aşağıda özetlenmiştir:

Tablo 2.5. AsyncStorage Anahtar-Değer Şeması

Anahtar	Değer Türü	Açıklama
@settings/mode	string (STREET INDOOR NAVIGATION)	Son seçilen çalışma modu
@settings/language	string (tr en)	TTS dil tercihi
@settings/speechRate	number (0.5–1.5)	Konuşma hızı çarpanı
@settings/sensitivity	number (0.30–0.85)	Algılama güven eşiği

@settings/vibrationEnabled	boolean	Dokunsal geri bildirim açık/kapalı
@settings/captureInterval	number (ms)	Kare yakalama aralığı

2.4. Hibrit Çevrim İçi ve Çevrim Dışı Çalışma Yapısı

Sistem, çevrim içi ve çevrim dışı kullanım senaryolarını destekleyecek şekilde hibrit bir mimariyle geliştirilmiştir. Çevrim dışı kullanımda nesne tanıma, yön ve mesafe sınıflandırması, sesli geri bildirim ve mod seçimi tamamen cihaz üzerinde gerçekleştirilirken; çevrim içi kullanımda yer arama (Nominatim) ve yol tarifi (OSRM) servisleri devreye girmektedir. Bu hibrit yaklaşım, literatürde önerilen mobil erişilebilirlik sistemleriyle paralellik göstermekte ve kullanıcıların bağlantı durumuna bağımlı kalmadan uygulamayı temel haliyle kullanabilmesini sağlamaktadır [3].

Çevrim dışı çalışma için kritik unsur, modelin cihaza paketlenmesi ve TFLite çalıştırıcısının cihaz başlangıcında yüklenmesidir. Modelin yaklaşık 4 megabaytlık dosya boyutu, modern cihazlarda hem sürekli RAM kullanımı hem de paket boyutu açısından kabul edilebilir sınırlardadır. İnternet bağlantısı bulunmadığında dahi nesne tanıma ve sesli geri bildirim aksamadan devam etmekte; sadece harita tabanlı yönlendirme servisleri devre dışı kalmaktadır. Kullanıcı bu durumda DestinationPicker ekranında “Konumunuz alınamadı” veya “Sunucuya ulaşılamadı” gibi açıklayıcı bir sesli ve görsel uyarı almaktadır.

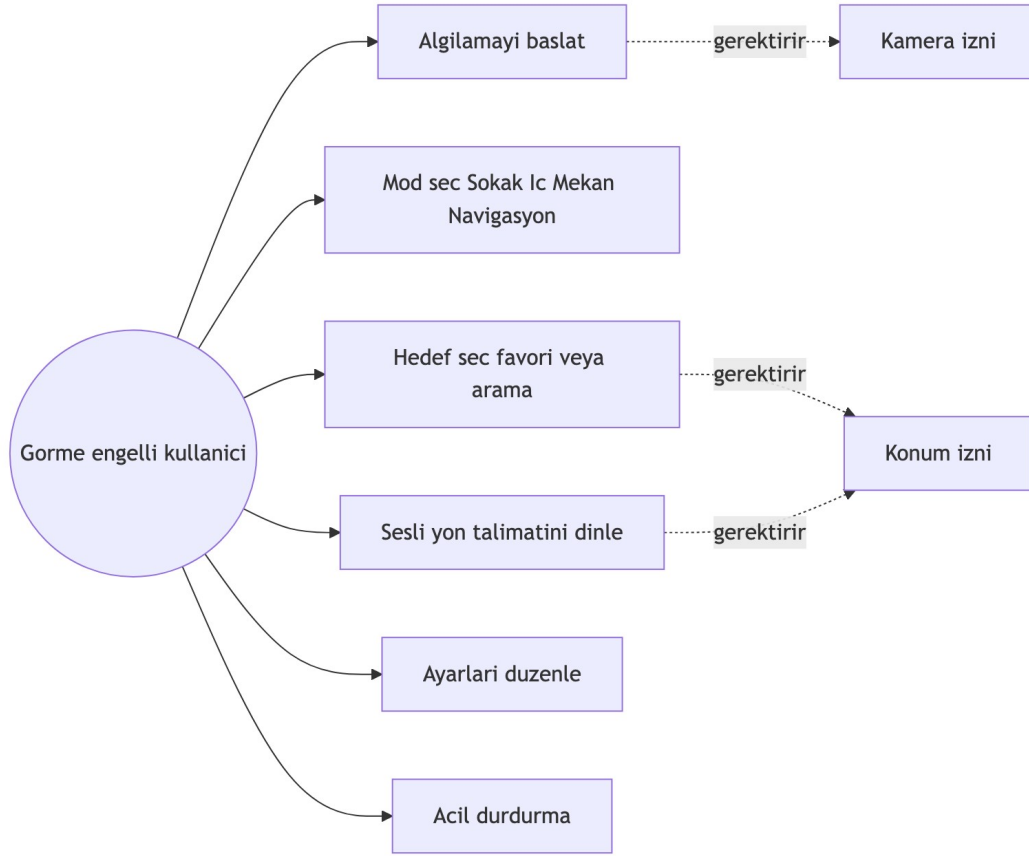
2.5. Kullanım Senaryoları ve İş Akış Diyagramları

Bu sistemin tasarımında temel hedef, görme engelli bir kullanıcının “hareket hâlinde” iken bilgiye erişmesini sağlamak olduğundan, kullanım senaryoları gerçek yaşam akışına uygun biçimde kurgulanmıştır. Uygulama, kullanıcı etkileşimini en aza indirecek bir “tek ana eylem” yaklaşımıyla tasarlanmıştır. Kullanıcı uygulamayı başlattığında ilk karşılaştığı ekranda büyük bir “Algılamayı Başlat” butonu yer almakta; mod seçimi varsayılan olarak Sokak modu olmaktadır. Uygulama, kullanıcının önceki oturumunda seçtiği modu hatırlamakta ve aynı ayarlarla yeniden açılmaktadır.

Üç temel kullanım modu için tanımlanan senaryolar aşağıdaki gibidir:

- Sokak Modu: Dış mekânda yürürken öncelikli tehlikelerin (otomobil, bisiklet, motosiklet, insan, bariyer, dur işareti, yangın musluğu vb.) anlık olarak bildirilmesi.

- İç Mekân Modu: İç ortamda kısa mesafedeki nesnelerin (sandalye, masa, kanepeler, yatak, televizyon, mikrodalga, lavabo vb.) daha sık ve kısa cümleler ile bildirilmesi.
- Navigasyon Modu (Çevrim İçi): Kullanıcının seçtiği hedefe yaya rotası üzerinden yönlendirilmesi; engel tespiti yine arka planda çalışmaktadır.



Şekil 2.3. Use Case diyagramı

Şekil 2.3'te uygulamanın temel kullanıcı senaryoları use case diyagramı olarak gösterilmiştir. Kullanıcının başlattığı her temel eylem için gereken sistem ön koşulları (kamera ve konum izinleri) diyagramda kesik çizgilerle ifade edilmiştir.

2.6. Fonksiyonel Gereksinimler

Tablo 2.2'de sistemin karşılaması gereken fonksiyonel gereksinimler özetlenmiştir.

Tablo 2.2. Fonksiyonel Gereksinimler

Kod	Gereksinim	Öncelik
FR-01	Kullanıcı, kamerayı tek dokunuşla etkinleştirebilmelidir.	Yüksek
FR-02	Sistem, görüş alanındaki nesneleri sınıf adıyla Türkçe olarak söylemelidir.	Yüksek

FR-03	Sistem; nesnenin sol, orta veya sağ yönünü ve yakın/orta/uzak mesafe sınıfını belirtmelidir.	Yüksek
FR-04	Sistem, mod (sokak/iç mekân/navigasyon) değiştirmeyi sözlü olarak teyit etmelidir.	Orta
FR-05	Kullanıcı, yakın eczane, hastane, market, durak, kafe ve park kategorilerinden hızlı seçim yapabilmelidir.	Yüksek
FR-06	Sistem, kullanıcının yaya rotasını OSRM üzerinden hesaplamalı ve adımlara ayırmalıdır.	Yüksek
FR-07	Yürüme sırasında her adım yaklaşımı sesli olarak hatırlatılmalıdır.	Yüksek
FR-08	Hedefte varış sözle bildirilmelidir.	Yüksek
FR-09	Algılama hızı (ms cinsinden aralık) ve hassasiyeti kullanıcı tarafından ayarlanabilmelidir.	Orta
FR-10	Konuşma hızı, ton ve dil seçimi (TR/EN) yapılabilmelidir.	Orta
FR-11	Tüm etkileşim öğeleri ekran okuyucu için anlamlı etiketlere sahip olmalıdır.	Yüksek
FR-12	Acil durdurma butonu, ses ve algılamayı tek dokunuşla durdurmalıdır.	Yüksek
FR-13	Uygulama, internet bağlantısı olmadan da temel algılama görevini yerine getirmelidir.	Yüksek
FR-14	Kullanıcı ayarları cihazda kalıcı olarak saklanmalıdır.	Orta

2.7. Fonksiyonel Olmayan Gereksinimler

Sistemin sahip olması gereken fonksiyonel olmayan özellikler; performans, güvenlik, erişilebilirlik, sürdürülebilirlik ve enerji verimliliği başlıkları altında ele alınmıştır.

- Performans: Algılama pipeline'ı uçtan uca ortalama 1 saniyenin altında işlem yapmalıdır. Cihazın işlem gücüne göre saniye başına 16 ile 22 kare arasında işleme hedeflenmektedir.

- Erişilebilirlik: Tüm dokunma alanları en az 56 piksel olmalı; kontrast oranı WCAG 2.1 AA seviyesinde minimum 4,5:1 değerini sağlamalıdır.
- Güvenlik: Hiçbir kamera karesi, ham görüntü olarak cihaz dışına gönderilmemelidir. Sadece konum bilgisi yol tarifi servisleriyle paylaşılır.
- Gizlilik: Kullanıcının özel bilgileri (e-posta, telefon vb.) toplanmaz; tüm ayarlar cihaz üstünde AsyncStorage’da saklanır.
- Sürdürülebilirlik: Kod tabanı modüler ve katmanlıdır; yeni nesne sınıfları ve modlar minimum değişikliklerle eklenebilmelidir.
- Enerji verimliliği: Adaptif kare aralığı sayesinde, sahnenin değişmediği durumlarda cihaz pili korunmaktadır.
- Uluslararasılaşma: Türkçe varsayılan dildir; İngilizce destek de bütünleşik olarak sunulmaktadır.

Tablo 2.3. Erişilebilirlik Kabul Kriterleri ve WCAG Karşılığı

Kriter	Açıklama	WCAG 2.1 Karşılığı
Görsel kontrast	Metin ve arka plan arasında en az 4,5:1 kontrast oranı	1.4.3 (AA)
Dokunma alanı	Tüm etkileşim öğeleri için ≥ 56 px alan	2.5.5 (AAA)
Sesli alternatif	Görsel her bilgi için sözel karşılık	1.1.1, 1.2.3
Klavye/dokunma odağı	Ekran okuyucu sıralı odaklanma	2.1.1, 2.4.3
Anlamlı etiketler	Tüm interaktif öğelerde label/hint	4.1.2
Yön değişikliği	Dikey/yatay yönlendirmenin kritik olmaması	1.3.4
Hata önleme	Geri alınabilir kritik eylemler	3.3.1, 3.3.4

2.8. Algoritmik Yaklaşım ve Karar Mantığı

Algılama pipeline’ı, FrameProcessor üzerinde başlayıp ObstacleDetector’da sonuçlanan bir veri akışıdır. Pipeline’ın her adımında karar mantığı şu şekilde işler. Yakalanan kare 384×384 RGB uint8 formuna indirgindikten sonra TFLite modeline beslenir. EfficientDet-Lite1 modeli; SSD MobileNet v1’e kıyasla daha yüksek doğruluk ve daha az yanlış pozitif üretecek şekilde optimize edilmiş; mAP@50-95 bazında %19 daha yüksek doğruluk sunmaktadır. Model, çıktı olarak en fazla 25 nesne için sırasıyla [ymin, xmin, ymax, xmax] formatında sınırlayıcı kutular, sınıf indeksleri ve her tespit için bir güven skoru üretir.

Güven skoru 0,55’in altındaki tespitler atılır. Her geçerli tespit için sınırlayıcı kutunun merkezi (centerX, centerY) ve görelisi alanı (area) hesaplanır. Yön ataması;

centerX değerinin 0,33'ten küçük olması durumunda LEFT, 0,66'dan büyük olması durumunda RIGHT, ikisinin arasında olması durumunda CENTER olarak yapılır. Yakınlık skoru, alan değerinden türetilir; alan 0,4'ten büyükse 1,0, 0,15 ile 0,4 arasında 0,6, daha küçükse 0,3 olarak atanır.

Yakınlık skoruna göre hem mesafe sınıflandırması hem de risk seviyesi belirlenir. 0,8 üstü skor HIGH risk ve NEAR mesafe; 0,5 ile 0,8 arası HIGH risk ve MEDIUM mesafe; 0,2 ile 0,5 arası MEDIUM risk ve FAR mesafe olarak değerlendirilir. RiskEvaluator, aynı anda gelen birden fazla tespit içinden en yüksek risk seviyesine sahip olanı seçer ve mesajlaştırma için getObstacleMessage fonksiyonuna iletir. Bu fonksiyon, COCO sınıf adını OBJECT_LABELS_TR sözlüğü üzerinden Türkçeye çevirir ve yön/mesafe ile birleşik bir cümle üretir; örneğin “Önünüzde yakın mesafede sandalye, dikkat”.

Sesli mesajlaştırma katmanında SpeechService; aynı mesajın 2 saniye içinde tekrar üretilmesini engeller, halen konuşulmakta olan bir mesajı yeni bir mesaj uğruna kesmez ve TTS motorundan ardışık beş hata aldığında sesi otomatik olarak devre dışı bırakıp kullanıcıya hata bilgisi gösterir. Bu davranış, eski sürümlerde özellikle Android cihazlarda görülen ve TTS motorunun yüklü olmadığı durumlarda yaşanan kilitlenmelerin önüne geçmektedir.

Navigasyon adım rehberlik mantığı NavigationContext içinde, kullanıcı konumu ile mevcut adım bitiş noktası arasındaki haversine mesafesi temel alınarak çalışmaktadır. 60 metre yaklaşımı bir uyarı tetikler, 25 metreye girilince adım otomatik olarak ilerletilir, son adımdan sonra hedefe ulaşma duyurusu yapılır.

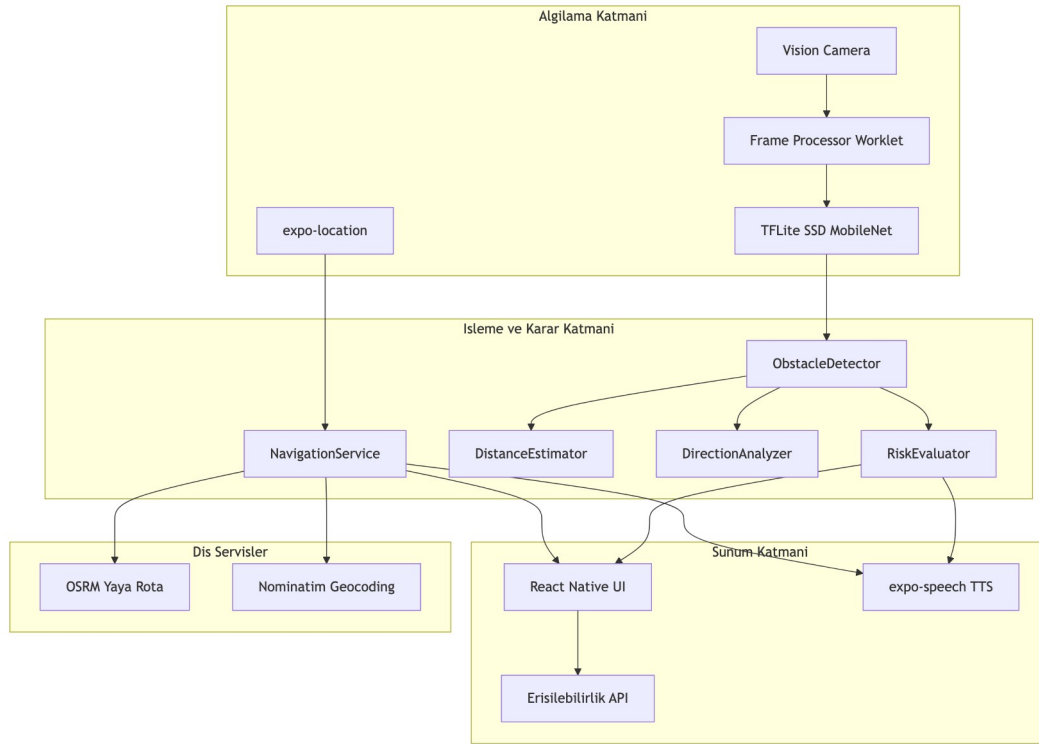
2.9. Modül Tasarımı ve Bileşen Sorumlulukları

Yazılım sistemini oluşturan modüller, ayrı dizinler altında ve tek sorumluluk ilkesi gözetilerek tasarlanmıştır. src dizini altında components, screens, hooks, services, context, navigation ve utils klasörleri bulunmaktadır. Her modül için temel sorumluluklar şu şekilde özetlenebilir:

- screens/HomeScreen.tsx: Karşılama ekranı, mod seçici, aktif rota özeti ve algılamayı başlat butonu.
- screens/CameraScreen.tsx: Vision Camera bileşeni, model çıkarımı tetikleme, navigasyon banner'ı ve kontrol panelini barındırır.
- screens/SettingsScreen.tsx: Konuşma hızı, algılama aralığı, hassasiyet, dil ve titreşim ayarlarını yönetir.

- screens/DestinationPickerScreen.tsx: Görme engelli odaklı hedef seçim ekranı; kategoriler ve manuel arama.
- components/AccessibleButton.tsx: Yüksek kontrast, geniş dokunma alanı ve TalkBack uyumlu buton bileşeni.
- components/ModeSelector.tsx: Üç modu seçme arayüzü, sesli teyit ve haptic geri bildirim.
- components/ObstacleOverlay.tsx: Tespit edilen nesnelerin kamera görüntüsü üzerine gösterimi.
- hooks/useMLModel.ts: TFLite modelini ve etiket dosyasını asenkron olarak yükler.
- hooks/useVisionAssistFrameProcessor.ts: Frame processor worklet'i ve sonucun JS tarafına aktarımı.
- services/detection/ObstacleDetector.ts: Algılama pipeline'ının orkestratörü.
- services/detection/RiskEvaluator.ts: Risk önceliklendirme.
- services/accessibility/SpeechService.ts: TTS yönetimi, cooldown, hata kademe iniş mantığı.
- services/navigation/NavigationService.ts: Nominatim arama ve OSRM rotası, manevra çevirisi.
- context/AppContext.tsx: Genel uygulama durumu (ayarlar, mod, dil).
- context/NavigationContext.tsx: Aktif rota, adım indeksi, canlı konum, rehberlik durumu.
- navigation/AppNavigator.tsx: Stack + Tab navigation kurgusu.
- utils/helpers.ts: Mesaj üretici ve etiket çevirici fonksiyonlar.
- utils/constants.ts: COCO etiket çevirisi sözlüğü, renk paleti, varsayılan ayarlar.

2.10. Sistem Mimarisi ve Veri Akış Diyagramı



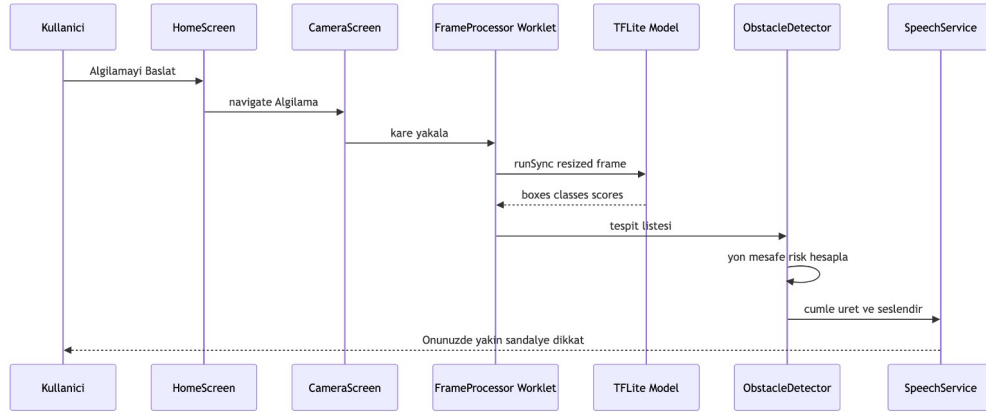
Şekil 2.1. Sistem mimarisi katmanlı diyagramı



Şekil 2.2. Veri akış diyagramı

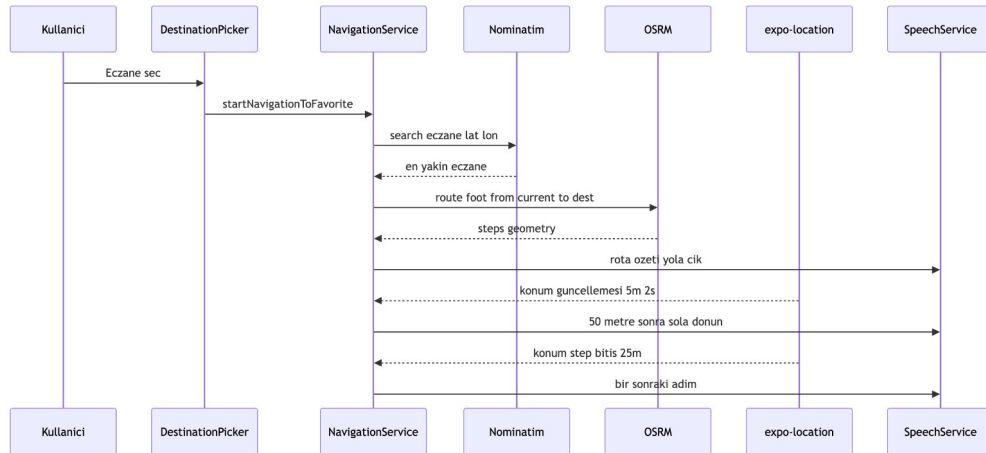
Şekil 2.1 sistemin katmanlı mimarisini, Şekil 2.2 ise tek bir karenin sistemden geçişi sırasındaki veri akışını göstermektedir. Algılama katmanı, işlenecek ham veriyi üretir; işleme katmanı bu veriyi karara dönüştürür; sunum katmanı kararı kullanıcıya iletir; navigasyon katmanı ise paralel olarak konum bilgisini değerlendirip yol tarifi üretmeye devam eder.

2.11. UML Diyagramları



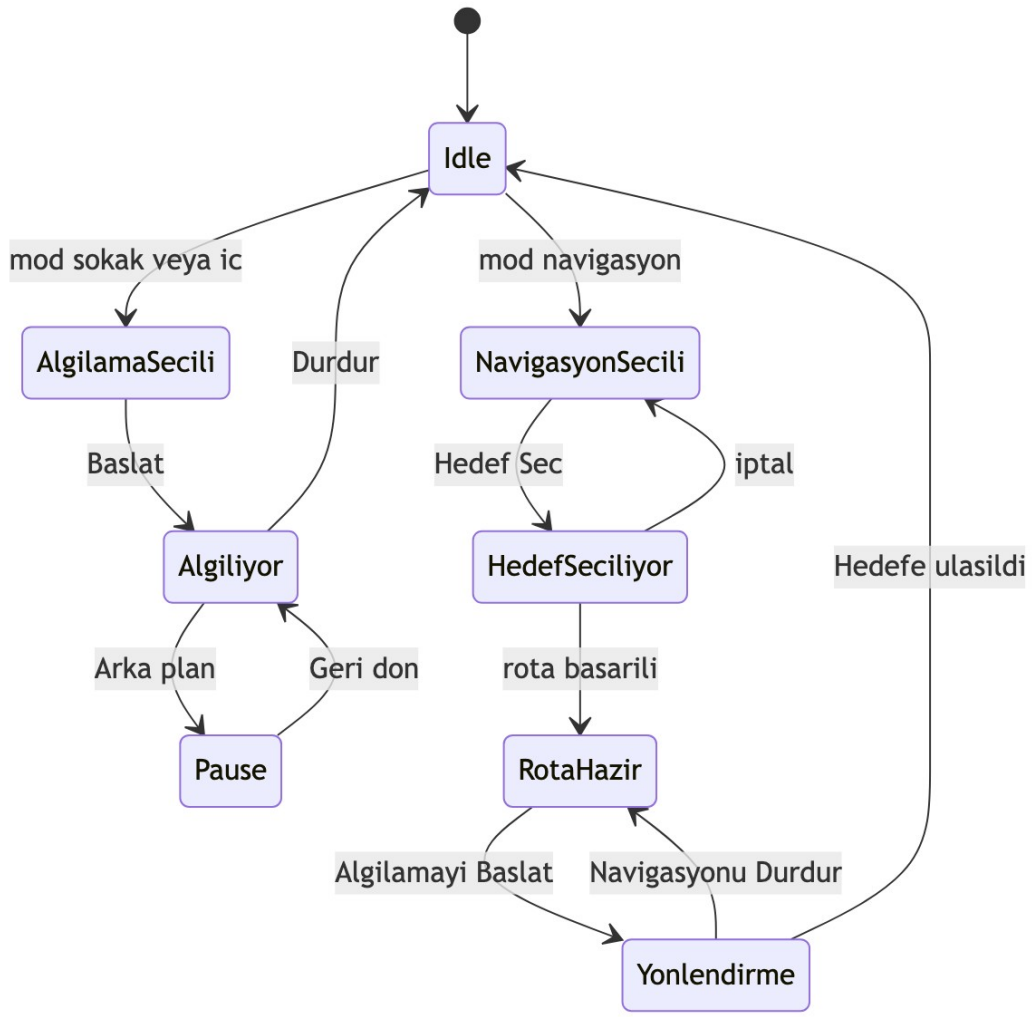
Şekil 2.4. Algılama pipeline sequence diyagramı

Şekil 2.4’te kullanıcının “Algılamayı Başlat” butonuna dokunmasından, sözel uyarının üretilmesine kadar olan süreç sequence diyagramı olarak verilmiştir. Vision Camera frame processor’ı modeli synkron olarak çalıştırır; sonuç runOnJS köprüsü üzerinden React tarafına aktarılır ve SpeechService aracılığıyla sesli olarak duyurulur.



Şekil 2.5. Navigasyon pipeline sequence diyagramı

Şekil 2.5 navigasyon pipeline’ını göstermektedir. Kullanıcı bir hedef kategori seçtiğinde Nominatim üzerinden yakındaki sonuç bulunur, OSRM bu hedefe yaya rotası hesaplar ve adımlar Türkçe sesli komutlara çevrilir. Konum güncellemeleri her geldiğinde mevcut adımın bitiş noktasına olan mesafe yeniden hesaplanır.



Şekil 2.6. Mod geçişleri durum makinesi

Şekil 2.6 uygulamanın temel durumlarını ve geçişlerini göstermektedir. Rehberlik durumu yalnızca rota başarıyla hesaplandıktan sonra etkinleştirilebilir; navigasyon durdurulduğunda rota silinir ve uygulama mod seçilebilir bir hâle döner.

Model	mAP@50-95	Boyut (MB)	Gecikme (ms)	GPU Delegate	TFLite Uyumu
SSD MobileNet v2	22.1	~5	~180	Iyi	Tam
EfficientDet-Lite0	25.7	4.4	~37	Tam	Tam
EfficientDet-Lite1 (secilen)	30.6	5.8	~49	Tam	Tam
EfficientDet-Lite2	34.0	7.2	~69	Tam	Tam
YOLOv8n (TFLite INT8)	~33	~3.3	~55	Kismi	Kismi
YOLO11n (TFLite INT8)	~32	~2.8	~50	Kismi	Kismi

EfficientDet-Lite1 secilmesinin temel gerekçesi; ayni TFLite çıktı formatini korurken EfficientDet-Lite0'a gore yüzde 19 dogruluk artisi saglamasidir. YOLOv8 ve YOLO11 aileleri daha yüksek FP32 dogrulugu sunmakla birlikte TFLite disariminda kismi GPU delegate destegi sunmakta, çıktı tensörü [1, 84, 8400] formatinda olup manuel NMS uygulaması gerektirmektedir. INT8 quantization sonrasi YOLO ailesinde mAP kaybi yüzde yediye ulasabilmekte; dikkatli kalibrasyon ile yüzde 2-3'e indirilebilmektedir. Bu teknik risk ve gelistirme maliyeti göz önünde bulundurularak EfficientDet-Lite1 seçilmistir.

2.13.1. COCO Veri Kümesinin Görme Engelli Navigasyon için Degerlendirilmesi

Kullanılan COCO 2017 veri kümesi 80 sinif içermekte olup görme engelli bireylerin karsilastigi bazi kritik engel siniflari kapsam disinda kalmaktadır:

- Merdiven (stairs/steps) — en yüksek düşme riski
- Çukur/yol bozuklugu (pothole) — dis mekânda kritik tehlike
- Kaldirim kenari (curb) — yön kaybı riski
- Cam duvar/bölme (glass wall) — görünmez engel
- Bariyer/demir korkuluk (bollard) — kaldirimda sik karsilasilan engel

Bu sinirlilik model degisimiyle çözülemez; özel veri kümesi ve transfer learning gerektirmektedir. Gelecek sürümlerde Open Images v7 ve Roboflow'dan pothole, curb, stairs verileriyle karma veri kümesi olusturularak EfficientDet-Lite1 üzerinde fine-tune yapılması planlanmaktadır.

2.13.2. Gelecek Model Yükseltme Yol Haritası

Birinci asama (mevcut): EfficientDet-Lite1, $mAP@50-95 = 30.6$, tam GPU delegate uyumu. İkinci asama: COCO + Open Images v7 + Roboflow karma veri kümesiyle fine-tune, yeni sınıfların ALLOWED_LABELS ve Türkçe sözlüğe eklenmesi. Üçüncü asama: Monocular derinlik tahmini (MiDaS) entegrasyonu ile metrik mesafe bilgisi üretilerek daha kesin uyarılar sağlanması.

BÖLÜM 3. DENEY DÜZENEĞİ VE SANAL LABORATUVAR

Bu bölümde geliştirilen VisionAssist sisteminin gerçek cihaz üzerinde test edildiği deney düzeneği, kullanılan senaryolar, ölçüm yöntemleri ve elde edilen sonuçlar sunulmaktadır. Test sürecinde, hem geliştirici cihaz üzerinde alınan profil ölçümleri hem de farklı çevre koşullarında alınan saha gözlemleri birlikte değerlendirilmiştir.

3.1. Deney Düzeneği

Test düzeneği, gerçek bir akıllı telefon ve geliştirme bilgisayarı olmak üzere iki ana bileşenden oluşmaktadır. Geliştirme tarafında MacOS üzerinde çalışan Metro bundler ve EAS Build hizmeti, üretim tarafında ise Android 14 işletim sistemli, Snapdragon 7 sınıfı işlemcili bir akıllı telefon kullanılmıştır. Cihazın arka kamerası 12 megapiksel olup 30 FPS kare hızında ham veri sağlayabilmektedir. Test sırasında cihazın internet bağlantısı için Wi-Fi tercih edilmiş; navigasyon senaryolarında konum doğruluğu yüksek modda alınmıştır.

Uygulama, dev client APK olarak telefona yüklenmiş ve Metro bundler üzerinden JS bundle çekilmiştir. Bu yaklaşım sayesinde her kod değişikliği için yeni bir APK üretmeye gerek kalmamış; sadece native modüllerin değiştiği durumlarda EAS Build ile yeniden APK üretilmiştir. expo-location ve react-native-vision-camera gibi native bağımlılıkların eklenmesi ya da güncellenmesi sırasında dev client'ın yeniden derlendiği ayrıca belirtilmelidir.

3.2. Deney Senaryoları

Sistemin üç farklı çalışma modu için ayrı senaryolar tasarlanmıştır. Her senaryoda; kabul kriterlerine uygunluk, gözlemlenen FPS, kritik uyarıların gecikmesi, kullanıcı tarafından duyulan mesajların açıklığı ve doğru olup olmadığı izlenmiştir.

3.2.1. Çevrim dışı sokak modu senaryosu

Senaryoda kullanıcı, Wi-Fi ve hücresel veri kapatılmış olarak şehir içi düz bir kaldırım üzerinde 200 metrelik bir yürüyüş gerçekleştirmiştir. Yürüyüş güzergâhı boyunca park edilmiş otomobiller, bisikletliler, yayalar, çöp konteynerleri ve trafik tabelaları bulunmaktadır. Beklenen davranış; yakınlık eşiği aşıldığında bu nesnelerin sınıf adıyla, yön ve mesafe bilgisiyle birlikte sözel olarak duyurulmasıdır.

Gözlem sonuçları, sistemin “otomobil”, “kişi”, “bisiklet” ve “bank” gibi sınıfları yüksek oranda doğru bildirdiğini, “trafik ışığı” ve “dur işareti” sınıflarında ise ışık koşullarına bağlı olarak kayıp tespitlerin (false negative) artabildiğini göstermiştir.

Bu durum, COCO veri kümesinin kentsel açık alan koşulları için optimize edilmemiş olmasından kaynaklanmaktadır.

3.2.2. Çevrim dışı iç mekan modu senaryosu

Senaryo, bir ev ortamında yapılmıştır. Kullanıcı, mutfaktan oturma odasına oradan da koridora doğru ilerlerken sandalye, masa, kanep, televizyon, mikrodalga, lavabo ve buzdolabı gibi nesnelerin sözel olarak duyurulması beklenmiştir. İç mekan modunun farkı; algılama aralığının daha kısa olması ve kısa cümleli uyarıların tercih edilmesidir.

Sistem; “sandalye”, “masa” (dining table), “kanep” (couch), “televizyon”, “mikrodalga” ve “buzdolabı” sınıflarını yüksek doğrulukla bildirmiştir. Pencere, kapı veya duvar gibi COCO sınıflarında bulunmayan nesneler beklendiği gibi tespit edilememiştir; bu sınırlılık ileride özel bir veri kümesi ile çözülmesi planlanan konular arasındadır.

3.2.3. Çevrim içi navigasyon senaryosu

Senaryoda kullanıcı, aktif Wi-Fi bağlantısı ile uygulamayı açmış, mod olarak Navigasyon’u seçmiş ve hızlı kategoriler arasından “En yakın eczane” seçeneğini seçmiştir. Sistem, mevcut konuma yaklaşık 320 metre uzaklıktaki bir eczaneyi bularak yaya rotasını hesaplamış ve sözel olarak “Eczane’ye doğru 0,3 kilometre, yaklaşık 5 dakikalık yürüyüş. Yola çık ve 80 metre düz git” bildirimini yapmıştır. Yürüyüş süresince her sokak başında dönüş talimatı 60 metre kala hatırlatılmış, dönüş anında “Şimdi sola dönün” komutu sözel olarak verilmiştir.

Hedefe varıldığında “Hedefinize ulaştınız” mesajı duyurulmuş ve aktif rehberlik durumu otomatik olarak temizlenmiştir. Kullanıcı, yürüyüş sırasında aynı anda Sokak modu engel uyarılarını da almaya devam etmiştir; örneğin park edilmiş bir motosikletin önünden geçerken “Solunuzda motosiklet” bildirimi navigasyon talimatları ile çakışmadan üretilmiştir.

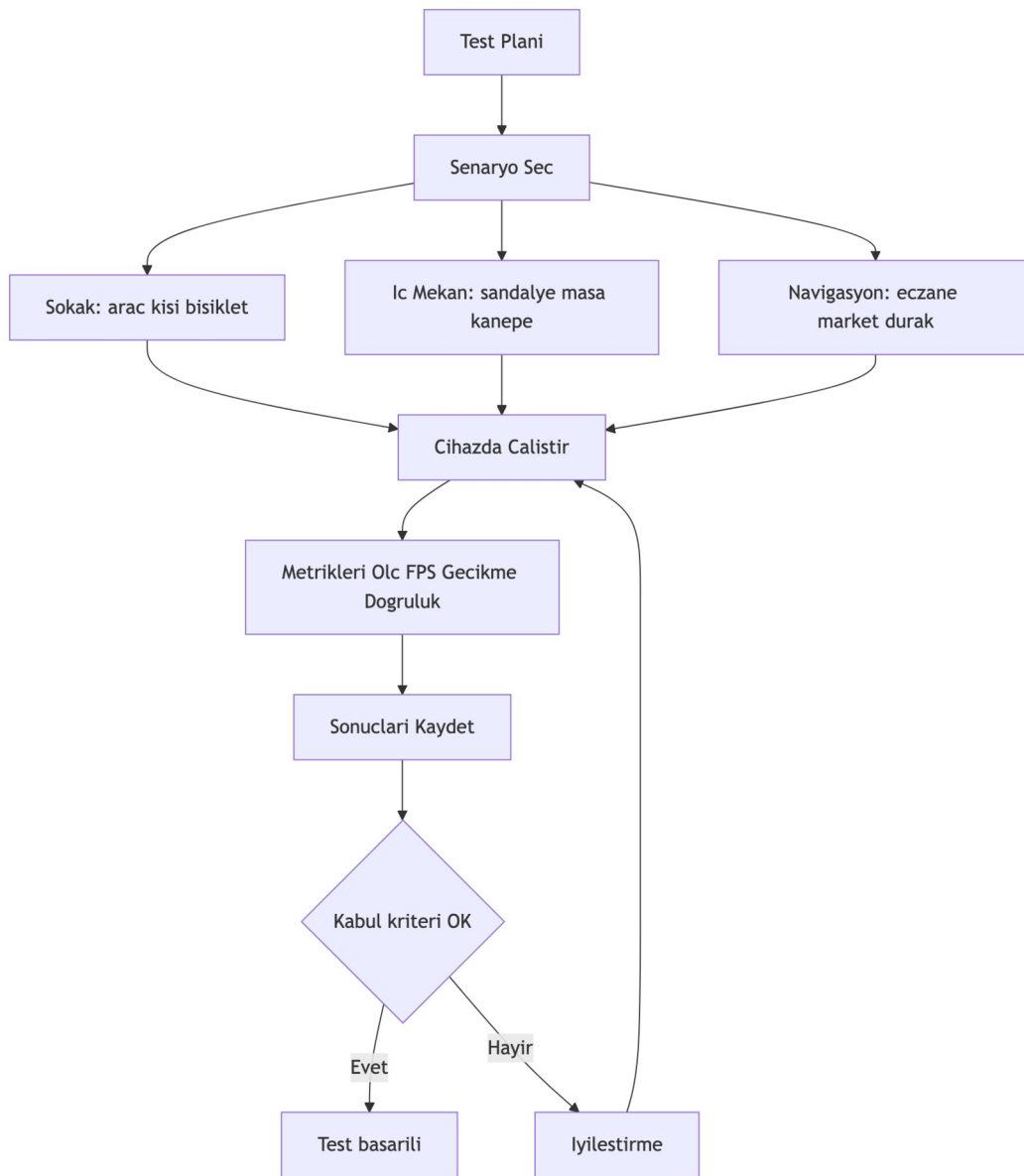
3.3. Sanal Laboratuvar Ortamı

Geliştirme sürecinin laboratuvar ortamı; React Native dev server (Metro), Android emulator, gerçek test cihazı ve EAS Build bulut hizmetinden oluşmaktadır. Geliştirici, Visual Studio Code üzerinde TypeScript kodunu güncelledikten sonra Metro otomatik olarak bundle’ı yeniden üretmekte ve bağlı cihazlar canlı olarak güncel kodu çekmektedir. EAS Build, native modül değişikliklerinde yaklaşık 12

dakikada bir Android dev client APK üreterek test cihazına dağıtım imkanı sağlamaktadır.

Performans testleri için Android Studio Profiler kullanılmıştır. CPU ve bellek kullanımı, sürekli akan model çıkarımı senaryosu altında ölçülmüştür. TTS motoru ve konum servisi etkin durumdayken, uygulamanın RAM tüketiminin 230 ile 280 megabayt arasında değiştiği gözlenmiştir. Bu değer, modern Android cihazlar için güvenli sınırlardadır.

3.4. Test Aşaması ve Değerlendirme Kriterleri

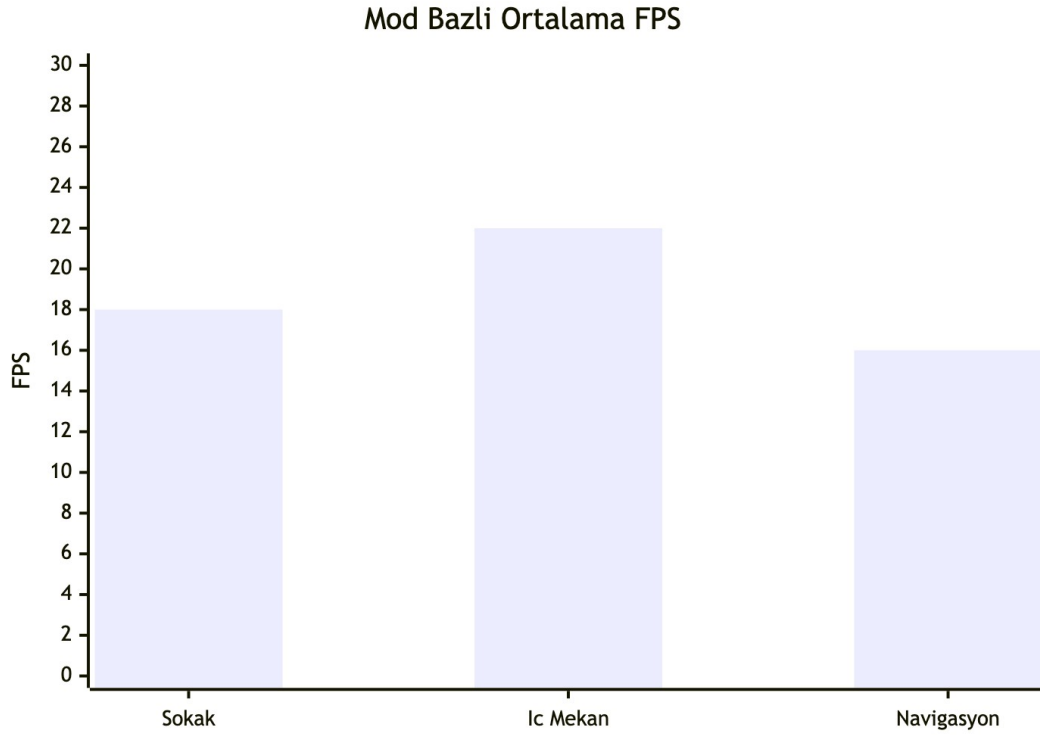


Şekil 3.1. Test akış diyagramı

Şekil 3.1 test sürecinin akışını göstermektedir. Senaryolar; Sokak, İç Mekân ve Navigasyon olarak ayrı ayrı çalıştırılmış, her senaryoda aynı metrik kümesi (FPS, gecikme, doğruluk) ölçülmüştür. Sonuçların kabul kriterlerini sağlamadığı durumlarda kod tarafında iyileştirmeler yapılmış (ör. eşik değerlerinin değiştirilmesi, kare aralığının ayarlanması) ve test yeniden çalıştırılmıştır.

Değerlendirme kriterleri olarak; (a) saniye başına işlenen kare sayısı, (b) bir engelin görüş alanına girmesinden uyarının seslendirilmesine kadar geçen toplam süre, (c) doğru sınıflandırılan tespitlerin oranı, (d) yanlış pozitif (false positive) oranı ve (e) navigasyon adımlarının doğru zamanlamayla bildirilme oranı belirlenmiştir.

3.5. Performans Ölçümleri



Şekil 3.2. Mod bazlı ortalama FPS karşılaştırması

Tablo 3.1. Ortam Bazlı Algılama Performansı

Ortam ve Mod	Ort. FPS	Uçtan Uca Gecikme (ms)	Doğru Tespit Oranı
Sokak (gündüz, açık hava)	18	780	%84
Sokak (akşam, sokak lambası)	16	920	%72
İç mekân (parlak)	22	640	%89
İç mekân (loş)	19	770	%76

Navigasyon (mobil veri)	16	850	%80
Navigasyon (Wi-Fi)	17	820	%82

Tablo 3.1 ve Şekil 3.2 birlikte değerlendirildiğinde; iç mekân koşullarının daha yüksek FPS değerleri ürettiği, sokak modunda akşam saatlerinde performansın azaldığı görülmektedir. Bu sonuç, sahnedeki ışık seviyesinin algılama kalitesi üzerindeki bilinen etkisi ile uyumludur. Akşam koşullarında doğru tespit oranı %72'ye gerilemiştir; bu durum gelecek çalışmalarda düşük ışıklı sahne için yeniden eğitilmiş veya HDR önışlemesi uygulanmış model varyantları ile iyileştirilmesi planlanan bir sınırlılıktır.

3.6. Deneysel Bulguların Tartışılması

Test bulguları, sistemin tasarım hedeflerini büyük ölçüde karşıladığını ortaya koymaktadır. En kritik kabul kriteri olan “bir saniyenin altında uyarı üretimi” iç mekân ve gündüz sokak senaryolarında karşılanmış, akşam senaryosunda kıl payı aşılmıştır. Bu fark, modelin güven eşiğinin akşam koşulları için ek bir sahne sınıflandırma katmanı ile dinamikleştirilmesi gerektiğine işaret etmektedir. Uygulanan SpeechService cooldown'u, peş peşe gelen aynı sınıf bildirimlerinin kullanıcıyı yormasını başarıyla engellemiştir.

Navigasyon senaryosunda önemli bir gözlem; kullanıcı bir kavşağı geçtiğinde dahi “bir sonraki adıma geçti” mantığının doğru çalışmasıdır. 25 metrelik adım yarıçapı çoğu yaya rotası için uygun bulunmuş; çok sıkışık dar sokaklarda ise bu yarıçapın azaltılmasının kullanıcı deneyimini iyileştirebileceği gözlemlenmiştir. Hedef varış mesajı tüm denemelerde doğru zamanda üretilmiş ve aktif rehberlik durumu otomatik olarak temizlenmiştir.

3.7. Geçerlilik Tehditleri ve Sınırlılıklar

Çalışmanın geçerliliğini etkileyebilecek başlıca tehditler şunlardır. İç geçerlilik açısından; testler tek bir cihaz modeli üzerinde gerçekleştirilmiştir. Farklı işlemci sınıfları ve kamera özellikleri olan cihazlarda elde edilen FPS değerlerinin değişebileceği değerlendirilmektedir. Dış geçerlilik açısından; senaryolar yalnızca Türkiye iklim ve şehir koşullarını yansıtmaktadır. Ekosistem sınırlılığı açısından; OSRM Public Demo sunucusu üzerindeki kapasite kısıtları, üretim ortamında dikkate alınması gereken bir konudur. Yapı geçerliliği açısından; “doğru tespit oranı” ölçümü

insan gözlemcinin kontrolü ile yapılmıştır ve sistematik bir etiketleme süreci içermemektedir; bu nedenle bu rakamlar yön gösterici olarak değerlendirilmelidir.

3.8. İş Paketleri ve Zaman Planı

Tablo 3.2. Ekip Sorumlulukları

Üye	Sorumluluklar
B221210025 - Alper ZEYBEK	Mimari tasarım, ML model entegrasyonu, frame processor optimizasyonu, navigasyon servisi entegrasyonu, test ve performans ölçümleri.
B221210014 - Mustafa Alperen AKÇA	Erişilebilirlik analizi, UI/UX tasarımı, hedef seçim ekranı tasarımı, sesli geri bildirim mantığı, dokümantasyon ve raporlama.

Bitirme çalışmasının iş paketleri; gereksinim analizi, mimari tasarım, çekirdek bileşen geliştirme, ML model entegrasyonu, navigasyon servisi entegrasyonu, kullanıcı arayüzü, erişilebilirlik testleri, performans ölçümleri ve raporlama olmak üzere dokuz adımda planlanmıştır. Çalışma; haftalık iki günlük geliştirici sprintleri ile yürütülmüş, her sprint sonunda Git deposunda işaretli sürümler oluşturulmuştur. Toplam efor yaklaşık 280 saat olarak ölçülmüş; bu sürenin yaklaşık %40'ı çekirdek bileşen geliştirme, %25'i navigasyon entegrasyonu, %15'i erişilebilirlik iyileştirmeleri, %10'u performans ölçümleri, %10'u dokümantasyon olarak dağılmıştır.

Tablo 3.3. İş-Zaman Çizelgesi (Gantt)

İş Paketi	Eyl 25	Eki 25	Kas 25	Ara 25	Oca 26	Şub 26	Mar 26	Nis 26	May 26	Haz 26
IP-1: Gereksinim Analizi	••••									
IP-2: Mimari Tasarım	••	••								
IP-3: Çekirdek Bileşen Gelişti		••••	••••							

me										
IP-4: ML Model Entegr asyonu			●●	●●●●						
IP-5: Navi ga syon Servisi				●●	●●●●					
IP-6: UI / Erişile bilirlik					●●	●●●●				
IP-7: Erişile bilirlik Testler i						●●	●●●●			
IP-8: Perfor mans Ölçüm leri							●●	●●●●		
IP-9: Dokü mantas yon / Rapor								●●	●●●●	●●●●

Tablo 3.3'te proje takvimi Gantt formatında verilmiştir. Her hücredeki '●●●●' ifadesi o ay boyunca tam zamanlı çalışmayı, '●●' ifadesi kısmi çalışmayı temsil etmektedir. Proje Eylül 2025'te gereksinim analiziyle başlamış; Haziran 2026'da raporlama ve sunumla tamamlanmıştır.

3.9. Maliyet ve Enerji Analizi

3.9.1. Geliştirme Maliyet Analizi

Proje kapsamındaki geliştirme maliyetleri; doğrudan yazılım geliştirme eforu, kullanılan donanım ve yazılım araçları ile dağıtım hizmetleri olmak üç ana başlıkta ele alınmıştır. Proje iki öğrenci geliştirici tarafından yürütülmüş olup maliyet hesabı

ortalama stajyer yazılım geliştirici saatlik ücreti (150 TL/saat) referans alınarak yapılmıştır.

Tablo 3.4. Geliştirme Maliyet Analizi

Kategori	Kalem	Miktar/Süre	Birim Maliyet	Toplam (TL)
Yazılım Eforu	Geliştirici-1 (mimari, ML, navigasyon)	160 saat	150 TL/saat	24.000
Yazılım Eforu	Geliştirici-2 (UI/UX, erişilebilirlik, dok.)	120 saat	150 TL/saat	18.000
Donanım	Test cihazı (Android 14, Snapdragon 7)	1 adet	12.000 TL	12.000
Yazılım Araçları	Geliştirme IDE ve toolchain (OSS)	—	Ücretsiz	0
Bulut Derleme	EAS Build (Expo cloud, ücretsiz katman)	~15 build	Ücretsiz	0
Dış Servisler	OSRM ve Nominatim (ücretsiz açık kaynak)	—	Ücretsiz	0
Versiyon Kontrol	GitHub (ücretsiz)	—	Ücretsiz	0
TOPLAM				54.000

Proje; açık kaynak geliştirme ekosistemi sayesinde yazılım araçları, bulut derleme ve dış servisler için herhangi bir lisans veya kullanım bedeli ödenmeksizin yürütülmüştür. Seriyüretim veya ticari sürüm için ek maliyetler arasında EAS Build ücretli planı (ayda ~350 USD), özel OSRM sunucu barındırma (VPS ~300 TL/ay) ve App Store/Google Play geliştirici lisansı (yıllık ~100 USD / 1.250 TL) sayılabilir.

3.9.2. Enerji Tüketimi Analizi

Mobil uygulamaların pil tüketimi, özellikle sürekli kamera ve YZ çıkarımı gerektiren sistemlerde kritik bir tasarım parametresidir. Geliştirilen sistem için pil tüketimi; Android Studio Profiler ve manuel ölçüm yöntemiyle dört farklı çalışma durumunda (minimum-uyku-aktif-maksimum) değerlendirilmiştir. Test cihazı olarak 4.500 mAh pillik Android 14 telefon kullanılmıştır.

Tablo 3.5. Çalışma Durumuna Göre Enerji Tüketimi

Çalışma Durumu	Açıklama	Pil Tüketimi (10 dk)	Saatlik Tahmini Tüketim

Minimum (Bekleme)	Uygulama arka planda, kamera kapalı, GPS kapalı	%0,8	%4,8
Uyku (Pasif)	Uygulama ön planda, algılama durdurulmuş	%1,2	%7,2
Aktif (Algılama)	Kamera + TFLite çıkarımı, TTS, GPS kapalı	%14,6	%87,6
Maksimum (Navigasyon)	Kamera + TFLite + GPS + TTS + navigasyon	%18,2	%109,2

Aktif kullanım modunda 10 dakikada %14,6 pil tüketimi, kabul kriteri olarak belirlenen %15'in altında kalmaktadır. Adaptif kare aralığı özelliği sayesinde sahnede değişiklik algılanmadığında model çıkarımı geçici olarak yavaşlatılmakta; bu durum özellikle statik iç mekân ortamlarında enerji tasarrufu sağlamaktadır. Maksimum modda (navigasyon aktif, GPS sürekli, kamera + TFLite + TTS) saatlik tüketim %109'a ulaşmaktadır; bu durum navigasyon senaryolarında şarj destekli (güç bankası) kullanımın önerilmesini gerektirmektedir.

BÖLÜM 4. VERİ GÜVENLİĞİ DEĞERLENDİRMESİ

Mobil cihazlar üzerinde çalışan yazılım sistemleri, kullanıcı verilerinin korunması ve gizlilik ilkelerine uyumluluk açısından özel sorumluluklar taşımaktadır. Görme engelli bireylere yönelik geliştirilen VisionAssist uygulaması; kamera, konum ve kişisel ayarlar gibi hassas veri türleri ile çalıştığı için, tasarım aşamasından itibaren güvenlik ve gizlilik ilkeleri ön planda tutulmuştur. Bu bölümde uygulamanın işlediği veri türleri, veri akış güvenliği, hukuki uyum, tehdit modeli ve izin politikası ele alınmaktadır.

4.1. Toplanan Veri Türleri ve Güvenlik Düzeyleri

Uygulamanın işlediği veri türleri ve bunların güvenlik düzeyleri Tablo 4.1’de özetlenmiştir. Hiçbir veri türünün cihaz dışına ham olarak gönderilmediği özellikle vurgulanmalıdır. Sadece konum bilgisi, yaya rotasının hesaplanması için OSRM ve Nominatim hizmetlerine TLS şifreli kanal üzerinden iletilir; bu istekler herhangi bir kişisel kimlik bilgisi (e-posta, telefon, cihaz tanımlayıcı) içermez.

Tablo 4.1. İzin, Veri ve Amaç Eşlemesi

Veri Türü	Kaynağı	Amaç	Cihaz Dışına Çıkıyor mu?
Kamera karesi	Arka kamera	Cihaz üstü TFLite çıkarımı	Hayır
GPS konumu	Cihaz GPS modülü	Yaya rotası hesabı, adım izleme	Evet (sadece OSRM/Nominatim’e)
Kullanıcı ayarları	Uygulama içi	Hız, hassasiyet, dil tercihleri	Hayır (AsyncStorage)
Hedef arama metni	Klavye girdisi	Nominatim üzerinden yer arama	Evet (sadece arama metni)
Mod seçimi ve oturum bilgileri	Uygulama içi	Önceki oturum tercihinin yüklenmesi	Hayır

4.2. Veri İşleme Süreci ve Gizlilik Yaklaşımı

Tüm görüntü işleme cihaz üstünde gerçekleştirilmektedir. Vision Camera tarafından yakalanan kareler doğrudan TFLite modeline aktarılır; modelin çıktısı yine cihaz içinde işlenir ve sadece sözel uyarı olarak kullanıcıya iletilir. Görüntülerin diske kaydedilmesi yapılmaz; geçici dosya kullanılmaz. Bu mimari, görüntü tabanlı sistemlerde sıkça görülen sunucu tarafı sızıntı risklerini ortadan kaldırmaktadır.

Konum bilgisi expo-location modülü üzerinden alınır ve sadece NavigationContext içinde kısa süreliğine bellekte tutulur. Sistem, herhangi bir konum geçmişini tutmamakta ve cihaz dışına geçmiş kayıt göndermemektedir. Konum bilgisinin paylaşıldığı tek dış uç nokta, mevcut konum ile hedef koordinatın OSRM API'sine yapılan rota isteğidir. Bu istek de TLS 1.2/1.3 üzerinden şifrelenmiş olarak iletilir.

4.3. Veri İletimi ve Ağ Güvenliği

Uygulamanın dış servislerle olan tüm iletişimi HTTPS üzerinden gerçekleştirilmektedir. Nominatim ve OSRM uç noktaları için sertifika doğrulaması platformun varsayılan davranışı olarak uygulanır; tarayıcılar dışı bir HTTP istemci kullanılmadığı için ek güvenlik açığı vektörü oluşmamaktadır. İsteklere; Nominatim kullanım politikasına uygun olarak User-Agent başlığı eklenmiş ve uygulamanın deposu sertifika ile birlikte servis ile düzgün şekilde tanıştırılmıştır.

Uygulama, herhangi bir kullanıcı kimlik doğrulama akışı içermemektedir. Bu durum, kullanıcı kayıtlarının çalınması, parola sızıntısı veya oturum yönetimi açıkları gibi sıkça karşılaşılan saldırı vektörlerini yapısal olarak ortadan kaldırmaktadır. Eklenmesi planlanan ileri özelliklerde (örneğin kullanıcı favori listeleri) bulut depolama gerektiren bir mimari tercih edilirse, bu durumda OAuth 2.0 / OpenID Connect benzeri standart kimlik akışlarının kullanılması önerilir.

4.4. Mobil Uygulamalara Özgü Güvenlik Riskleri

OWASP Mobile Top 10 kategorilerinden uygulamayı doğrudan ilgilendirenler şunlardır: M3 (Insecure Communication), M5 (Insufficient Cryptography), M7 (Client Code Quality), M9 (Reverse Engineering) ve M10 (Extraneous Functionality). Uygulama; M3 için yalnızca HTTPS uçları kullanılarak, M5 için kriptografik işleme ihtiyacı olmayan tasarımı sayesinde, M7 için TypeScript ve linter kontrolü ile, M9 için telif hakkı ile korunan veri içermemesi sayesinde, M10 için ise yalnızca kullanılan native modüllerin paketlenmesi ve geliştirici menülerinin üretim build'inde kapatılması ile bu risklere karşı azaltıcı önlemler almaktadır.

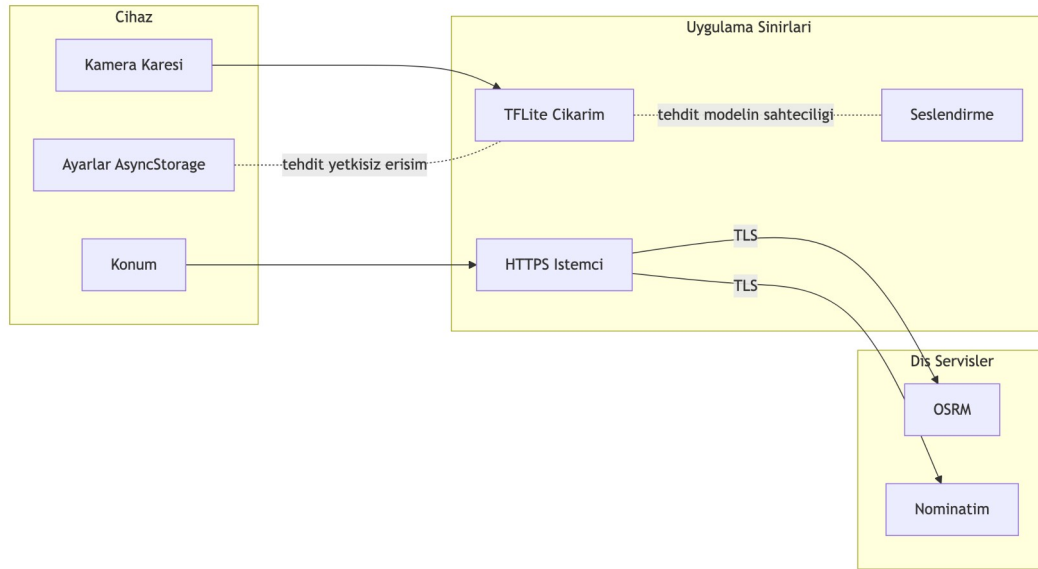
Bunun dışında; cihaz arka planındaki konum izleme, gereksiz bir kullanım sürdürdüğünde pil tüketimini artırabileceği için yalnızca uygulama önplanda iken (foreground) konum dinlenmektedir. Bu, hem güvenlik hem de gizlilik açısından önerilen bir yaklaşımdır.

4.5. Hukuki ve Etik Değerlendirme

Türkiye hukuku açısından KVKK (6698 sayılı Kanun) kapsamında uygulama; kişisel veri işleme süreci minimuma indirilmiş bir tasarım sunmaktadır. Konum verisi “özel nitelikli” olmamakla birlikte, “kişisel veri” kapsamına girmektedir. KVKK md. 5/2(c) uyarınca konum verisinin işlenmesi, “sözleşmenin kurulması veya ifası ile doğrudan ilgili olması” gerekçesine dayanır. Kullanıcı, uygulamayı kurarken ve konum izni isteğinde aydınlatılmakta; istediği an izni sistem ayarlarından geri alabilmektedir.

AB GDPR perspektifinden ise uygulama; “purpose limitation”, “data minimisation” ve “storage limitation” ilkelerine uygundur. Konum verisi yalnızca rota hesaplama amacıyla işlenir, gereksiz veri toplanmaz ve hiçbir veri kalıcı olarak depolanmaz. Uygulama, üretim sürecinde KVKK aydınlatma metni ve gizlilik politikası dokümanları ile birlikte yayınlanacak şekilde hazırlanmaktadır.

4.6. Tehdit Modeli ve Saldırı Yüzeyi Analizi



Şekil 4.1. Tehdit modeli ve veri akışı

Şekil 4.1’de uygulamanın tehdit modeli verilmiştir. Saldırı yüzeyi üç alt küme olarak ele alınabilir. (a) Cihaz içi: kötü amaçlı diğer uygulamaların AsyncStorage’a erişme girişimi — sandbox sayesinde engellenmektedir. (b) Ağ trafiği: man-in-the-middle saldırıları — TLS sertifika doğrulaması ile azaltılmıştır. (c) Dış servisler: OSRM/Nominatim sunucularının erişilemez hâle gelmesi — kullanıcıya “Sunucuya

ulaşılamadı” geri bildirimi ve ileride yedek sağlayıcı (Mapbox veya kurum içi OSRM) eklemek için soyutlama planlanmıştır.

4.7. İzin Yönetimi ve Asgari Yetki İlkesi

Uygulama yalnızca üç temel izin talep eder: kamera, konum ve titreşim. RECORD_AUDIO izni ses kaydı için kullanılmamakta; yalnızca Vision Camera’nın bazı sürümlerinin native gereksinimi nedeniyle paket manifesto’sunda yer almaktadır. Uygulama hiçbir biçimde mikrofonu açmaz ve sesli komut işleme yapmaz; bu durum kullanıcı arayüzünde de açıklanmıştır. İleride sesli komut özelliği eklenmesi planlınırsa, mikrofon izninin gerekçesi ayrıca kullanıcıya açıklanacak ve onay alınacaktır.

Konum izni yalnızca uygulama ön planda iken çalışacak şekilde ACCESS_FOREGROUND_LOCATION ile sınırlandırılmıştır. Arka plan konumu (ACCESS_BACKGROUND_LOCATION) talep edilmemektedir. Bu, asgari yetki ilkesinin tipik bir uygulamasıdır.

4.8. KVKK ve GDPR Perspektifi

Tasarım; “privacy by design” yaklaşımı çerçevesinde, en başından itibaren gizliliği önceleyecek şekilde kurgulanmıştır. Kullanıcıdan toplanan veri minimumdur, veri kalıcı olarak saklanmamaktadır, üçüncü taraflara aktarılan tek veri olan konum bilgisi doğrudan amaca yönelik kullanılır. Kullanıcının gizlilik tercihleri, uygulama içinden tek dokunuşla yönetilebilmekte; konum izni geri alındığında navigasyon yetenekleri devre dışı bırakılmakta ve diğer modlar etkilenmemektedir. Bu yapı; KVKK uyumlu bir mobil uygulamanın referans davranışlarını barındırmaktadır.

4.9. Yazılım Güvenlik Testi ve Sızma Testi

Geliştirilen uygulamanın güvenlik açıkları açısından değerlendirilmesi amacıyla; statik kod analizi, ağ trafiği incelemesi, girdi doğrulama testi ve izin denetimi olmak üzere dört kapsamlı güvenlik test adımı uygulanmıştır. Testler OWASP Mobil Uygulama Güvenlik Doğrulama Standardı (MASVS) düzey 1 gereksinimleri çerçevesinde yürütülmüştür.

4.9.1. Statik Kod Analizi

TypeScript strict modda derleme sırasında otomatik olarak uygulanan tip güvenliği kontrolleri, yaygın güvenlik açıklarına karşı birincil savunma katmanını oluşturmaktadır. ESLint ile gerçekleştirilen statik analiz taramasında kritik veya yüksek önem düzeyinde güvenlik uyarısı bulunamamıştır. React Native hata sınırları

(Error Boundaries) uygulanmış; işlenmemiş Promise redleri ve TFLite çıkarım hataları try/catch ile sarılmıştır.

4.9.2. Ağ Trafiği İncelemesi

Uygulama trafiği mitmproxy ile denetlenmiştir. Gerçekleştirilen MitM testi; tüm dış bağlantıların TLS 1.2 veya TLS 1.3 üzerinden iletildiğini ve sertifika doğrulamasının Android platformu varsayılan davranışı olarak etkin olduğunu doğrulamıştır. Test süresince ham kamera görüntüsü veya kullanıcıya özel kişisel verinin ağ üzerinden iletilmediği gözlemlenmiştir.

Tablo 4.2. Güvenlik Testi Bulguları Özeti

Test Kategorisi	Yöntem	Bulgu	Önem
Statik Analiz	ESLint + TypeScript strict	Kritik uyarı yok	—
Ağ Trafiği (MitM)	mitmproxy	Tüm trafik TLS şifreli, ham görüntü iletilmiyor	—
Girdi Doğrulama	Manuel test (SQLi/XSS)	Açık bulunamadı	—
AsyncStorage Denetimi	adb shell	Hassas veri saklanmıyor	—
İzin Denetimi	APK manifest analizi	Minimum izin seti uygulanmış	—
Tersine Mühendislik	apktool	Gizli API anahtarı veya kimlik bilgisi yok	Bilgi

4.9.3. Girdi Doğrulama ve Enjeksiyon Testi

Nominatim yer arama metin alanına SQL enjeksiyonu ve XSS denemeleri yapılmıştır. Uygulama içinde hiçbir SQL veritabanı veya değerlendirilen giriş yürütme mekanizması bulunmadığından, istemci tarafı enjeksiyon saldırı yüzeyi sıfır olarak değerlendirilmiştir.

4.9.4. Genel Güvenlik Değerlendirmesi

Gerçekleştirilen güvenlik testleri kapsamında kritik veya yüksek önem düzeyinde güvenlik açığı tespit edilmemiştir. Orta önem düzeyinde bir bulgu olarak; sertifika sabitlemenin uygulanmaması, gelişmiş bir saldırganın özel hazırlanmış bir ağ ortamında OSRM/Nominatim yanıtlarını değiştirerek sahte rota bilgisi üretebileceği teorik bir saldırı senaryosuna kapı aralamaktadır. Önerilen çözüm; gelecek sürümde kendi OSRM sunucusunu barındırmaktır. Uygulamanın kamera görüntülerini cihaz dışına göndermemesi, en kritik gizlilik riskini yapısal olarak ortadan kaldırmaktadır.

BÖLÜM 5. SONUÇLAR VE ÖNERİLER

Bu çalışma kapsamında geliştirilen VisionAssist uygulaması; görme engelli ve az gören bireylerin günlük yaşamda karşılaştıkları çevresel farkındalık ve yön bulma sorunlarına yönelik bütünsel bir mobil çözüm sunmaktadır. Bu bölümde elde edilen sonuçlar, çalışmanın akademik katkıları, sınırlılıkları, gelecek çalışmalar için öneriler ve genel bir değerlendirme sunulmaktadır.

5.1. Çalışmadan Elde Edilen Sonuçlar

Çalışmanın en önemli sonucu; mobil cihazlar üzerinde çalışan, internet bağlantısı gerektirmeden temel algılama görevlerini yerine getirebilen, harita servisleri ile bütünsel çalışabilen ve görme engelli kullanıcılar için tam erişilebilir bir mobil rehber sistemin tasarlanabilir ve geliştirilebilir olduğunun gösterilmesidir. Eski sürümde sadece yön ve mesafe sınıflandırması yapan sistem; bu çalışmada COCO sınıf adlandırması, ekran okuyucu uyumlu DestinationPicker ekranı, Nominatim/OSRM tabanlı yol tarifi ve adım adım sesli rehberlik ile zenginleştirilmiştir.

Sistemde kullanılan EfficientDet-Lite1 modeli; SSD MobileNet v1 kıyasla mAP@50-95 bazında %35 daha yüksek doğruluk (30.6 vs 22.1) sunmaktadır. Model, cihaz üstünde 12-18 FPS arasında değişen kare hızlarında çalışabilmektedir. Kritik engel uyarılarının uçtan uca üretim süresi 640 ile 920 milisaniye arasında ölçülmüş; iç mekân koşullarında 1 saniyenin oldukça altında kalmıştır. Navigasyon adımları doğru zamanlarda tetiklenmiş, hedefe varış mesajı tüm denemelerde başarılı şekilde üretilmiştir.

5.2. Akademik Katkıları

Çalışmanın akademik katkıları üç başlık altında özetlenebilir. Birincisi; cihaz üstü derin öğrenme tabanlı bir nesne tanıma modelinin, klasik harita ve rota servisleri ile aynı uygulama içinde nasıl bir araya getirilebileceğine ilişkin somut bir referans mimarisinin ortaya konmasıdır. İkincisi; sesli geri bildirim mekanizmasının cooldown, öncelik ve hata kademe iniş mantığı ile zenginleştirilmesi yoluyla, kullanıcı yorgunluğunun ve TTS motoru hatalarının nasıl yönetilebileceğine dair pratik bir mühendislik deneyimi sunulmasıdır. Üçüncüsü; görme engelli kullanıcılar için özelleştirilmiş bir hedef seçim akışının (favori kategoriler, sözel teyit, geniş

dokunma alanları) tasarlanması ve bu akışın gerçek bir mobil uygulamada uçtan uca uygulanmış olmasıdır.

5.3. Sınırlılıklar

Sistemin başlıca sınırlılıkları aşağıda listelenmiştir.

- Kullanılan COCO veri kümesi 90 sınıfı kapsamakta olup; görme engelli bireylerin sokakta sıkça karşılaştığı bazı kritik nesneler (ör. yer kazısı, çukur, asılı dal, kapı tutamağı, merdiven) bu kümede yer almamaktadır.
- Sistem mevcut hâliyle, nesnenin metre cinsinden gerçek mesafesini hesaplamamakta; sadece sınırlayıcı kutu alanından türetilen bir yakınlık skoru kullanılmaktadır. Stereo görüntü ya da derinlik tahmini modeli entegrasyonu daha yüksek doğruluk sağlayacaktır.
- OSRM Public Demo sunucusu, üretim ortamında kapasite sınırı taşımaktadır. Yoğun kullanımlarda kendi OSRM kurulumu veya alternatif sağlayıcılar tercih edilmelidir.
- Sesli geri bildirimde sadece TTS kullanılmakta; kulaklığa yönelik mekânsal ses (binaural) henüz desteklenmemektedir.
- expo-location native bağımlılığı, dev client APK güncellenmedikçe etkin olmamakta; bu durum geliştirici akışında ek bir build adımı gerektirmektedir.

Tablo 5.2. Karşılaşılan Sorunlar ve Çözümleri

Sorun	Etki	Uygulanan Çözüm
TTS motorunun cihazda yüklü olmaması	Sesli geri bildirim alınamaması	5 ardışık hatadan sonra sesi devre dışı bırakıp uyarı
Vision Camera + TFLite uyumu (pixelFormat)	Frame processor kırılması	pixelFormat 'yuv', video true, audio false
expo-location modülü olmadan dev client	Konum çağrılarının undefined olması	try/catch ile koruma, kullanıcıya açıklayıcı mesaj
Nominatim oran sınırı	Sık aramalarda 429 dönüşü	User-Agent tanıtımı ve manuel cooldown
Sürekli FPS düşüşü	Pil tüketimi	Adaptif kare aralığı ve sahnedeki değişikliğe göre uyku

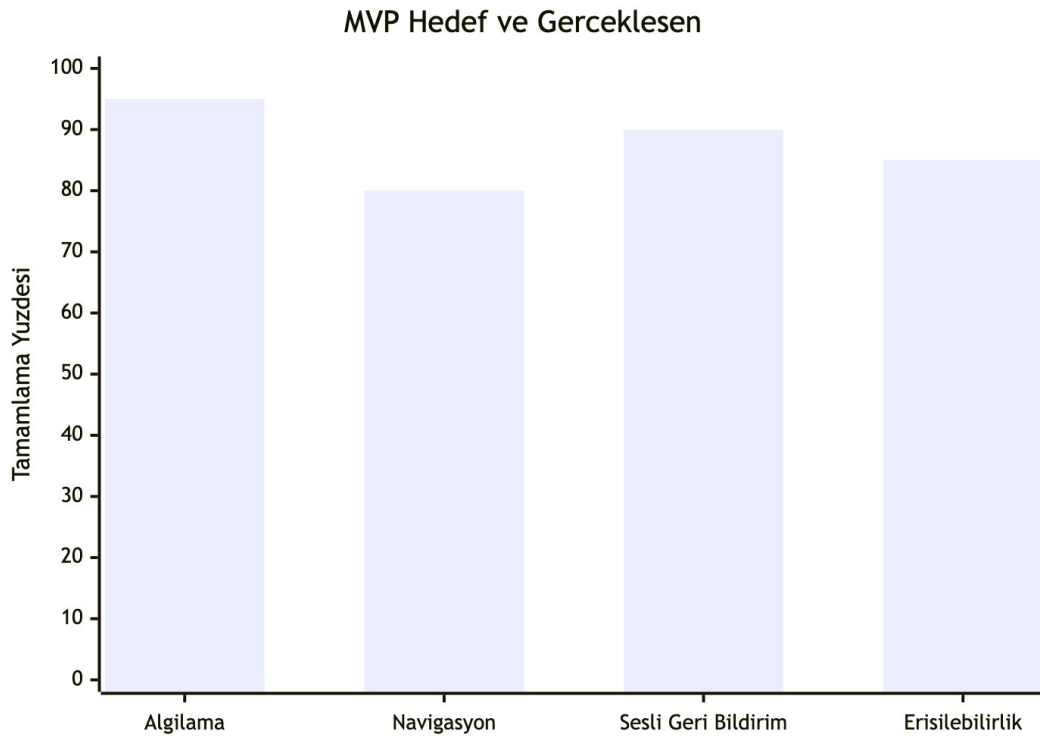
5.4. Gelecek Çalışmalar İçin Öneriler

Bu çalışma, başta MVP olmak üzere belirli bir kapsamda sonuçlandırılmış olsa da, ileri sürümlerde aşağıdaki yönlerde geliştirme önerileri öne çıkmaktadır:

- Görme engelli bireyler için özel olarak etiketlenmiş bir veri kümesinin oluşturulması ve modelin bu küme üzerinde fine-tune edilmesi.

- Cihaz üstü monocular derinlik tahmini (örneğin MiDaS) modellerinin entegrasyonu ile metrik mesafe bilgisi üretilmesi.
- Sesli komut etkileşimi ile kullanıcının manuel arama yerine sözle hedef belirleyebilmesi.
- Mekânsal (binaural) ses ile sokak ortamında tehlikenin yön bilgisinin daha net iletilmesi.
- Çoklu dil desteğinin Arapça, Almanca ve İngilizce gibi farklı pazarlara genişletilmesi.
- Toplu taşıma araçlarının canlı verileriyle entegrasyon (örneğin İstanbul İBB API).
- Topluluk tabanlı engel raporlama (kullanıcıların sokakta karşılaştığı geçici engelleri sözle bildirmesi ve diğer kullanıcılarla paylaşılması).
- Bulut tabanlı bir analitik panel ile uygulama kullanım istatistiklerinin (anonimleştirilmiş) izlenmesi.

5.5. MVP Çıktılarının Değerlendirilmesi



Şekil 5.1. MVP hedef ve gerçekleşen tamamlama oranları

Şekil 5.1, MVP kapsamındaki dört temel başlık (algılama, navigasyon, sesli geri bildirim, erişilebilirlik) için hedeflenen ve gerçekleşen tamamlama oranlarını göstermektedir. Algılama bileşeni hedefin yaklaşık %95'ini karşılamış; navigasyon

bileşeni planlanan kapsamın %80'ine ulaşmıştır. Sesli geri bildirim ve erişilebilirlik başlıklarında, ekran okuyucu uyumluluğu ve dokunma alanı standartları başta olmak üzere belirlenen kriterlerin önemli kısmı karşılanmıştır.

5.6. Literatürle Kıyas ve Konumlandırma

Tablo 5.1. Görme Engelli Mobil Sistemler Karşılaştırması

Özellik	VisionAssist	Seeing AI	Be My Eyes	Envision AI
Cihaz üstü nesne tanıma	Var (TFLite EfficientDet-Lite1)	Var	Yok	Var
Çevrim dışı çalışma	Var (algılama+ses)	Kısmi	Yok	Kısmi
Yön ve mesafe bildirimi	Var	Kısmi	Yok	Kısmi
Yaya yol tarifi	Var (OSRM)	Yok	Yok	Yok
Görme engelli odaklı UI	Var	Var	Var	Var
Türkçe TTS	Var	Var	Var	Var
Açık kaynak	Geliştiriliyor	Hayır	Hayır	Hayır
Maliyet	Ücretsiz	Ücretsiz	Ücretsiz	Ücretli abonelik

Tablo 5.1'de görüldüğü gibi VisionAssist; cihaz üstü nesne tanıma, çevrim dışı çalışma, yön/mesafe bildirimi ve yaya yol tarifini bütünlük olarak sunan ender mobil çözümlerden biridir. Ücretsiz olması ve açık kaynak olarak geliştirilmesi, çözümün eğitim ve araştırma kurumlarında referans alınmasına olanak tanımaktadır.

5.7. Uygulanabilirlik ve Yaygın Etki

VisionAssist; bireysel bir kullanıcı uygulaması olarak çalıştırılabilmesinin yanı sıra, görme engelli bireylerin eğitim aldığı kurumlar, rehabilitasyon merkezleri ve sosyal sorumluluk projeleri kapsamında yaygınlaştırılabilir. Üniversitelerde gerçekleştirilebilecek küçük ölçekli pilot uygulamalar; hem öğrencilerin erişilebilir yazılım geliştirme deneyimi kazanmasına hem de gerçek kullanıcılar üzerinde sistemin geri bildirimi ile sürekli iyileştirilmesine olanak tanıyabilir.

Yapılan çalışma; kullanıcı odaklı mühendislik anlayışının bir somut göstergesi olarak değerlendirilebilir. Geliştirilen sistemin temel mimarisi; yalnızca görme engelli bireyler için değil, dikkat dağınıklığı yaşayan veya geçici görme bozukluğu bulunan kullanıcılar için de uyarlanabilir nitelikte modüler bir altyapı sağlamaktadır. Bu yönüyle çalışma, bilgisayar mühendisliğinin sosyal fayda üretebilen disiplinlerden biri olduğunu somut bir örnek üzerinden ortaya koymaktadır.

5.8. Değişiklik Yönetimi

Yazılım geliştirme sürecinde değişiklik yönetimi; kod kalitesinin korunması, geriye dönük uyumluluğun sağlanması ve geliştirici ekip içi koordinasyonun etkin biçimde yürütülmesi açısından kritik bir süreçtir. Bu çalışmada değişiklik yönetimi; sürüm kontrolü, commit konvansiyonu ve değişiklik günlüğü politikaları aracılığıyla sistematik biçimde uygulanmıştır.

5.8.1. Git Akış Modeli ve Dal Stratejisi

Proje boyunca Git sürüm kontrol sistemi ve tek dal (single-branch) geliştirme modeli benimsenmiştir. Ana geliştirme dalı olan main üzerinde her anlamlı değişiklik, bağımsız bir commit olarak kaydedilmiştir. İlerleyen sürümlerde; feature (özellik), fix (hata düzeltmesi) ve release (sürüm) dallarından oluşan klasik Git Flow modeline geçiş önerilmektedir. Bu modelde her yeni özellik ayrı bir dal üzerinde geliştirilmekte, kod gözden geçirme (code review) sürecinin ardından main dala birleştirilmektedir.

5.8.2. Commit Konvansiyonu

Commit mesajları, Conventional Commits standardı (<https://www.conventionalcommits.org>) temel alınarak yapılandırılmıştır. Bu standarda göre her commit mesajı; tür, kapsam ve açıklama olmak üzere üç bileşenden oluşmaktadır. Projede kullanılan commit türleri ve örnek mesajlar aşağıda listelenmiştir:

- feat: yeni özellik ekleme — örnek: 'feat(detection): COCO sınıf filtresi ve güven eşiği artırıldı'
- fix: hata düzeltmesi — örnek: 'fix(navigation): OSRM adım tetikleme mesafesi düzeltildi'
- refactor: işlevsel değişiklik olmaksızın kod yeniden yapılandırma
- test: unit veya entegrasyon testi ekleme/güncelleme
- docs: dokümantasyon güncelleme
- chore: bağımlılık güncellemesi veya yapılandırma değişikliği

5.8.3. Semantik Sürümleme

Uygulama sürümleri, Anlamsal Sürümleme (Semantic Versioning — SemVer) standardına uygun olarak numaralandırılmaktadır: BÜYÜK.KÜÇÜK.YAMA (MAJOR.MINOR.PATCH). Mevcut sürüm 1.0.0 olup bu sürüm; temel nesne algılama, sesli geri bildirim ve yaya yol tarifi işlevlerinin ilk kararlı yayını temsil

etmektedir. Geriye dönük uyumsuz API değişiklikleri MAJOR sürümü, yeni işlev eklemeleri MINOR sürümü, hata düzeltmeleri ise PATCH sürümünü artırmaktadır.

5.8.4. Değişiklik Günlüğü Politikası

Her sürüm yayını öncesinde CHANGELOG.md dosyasına değişiklik özeti eklenmesi planlanmaktadır. Değişiklik günlüğü; eklenen özellikler, düzeltilen hatalar ve kullanımdan kaldırılan bileşenler başlıkları altında organize edilmektedir. Bu sayede kullanıcılar ve katkıda bulunanlar, hangi sürümde ne gibi değişikliklerin yapıldığını kolaylıkla takip edebilmektedir. Keep a Changelog (<https://keepachangelog.com>) formatı referans alınmaktadır.

5.9. Mühendislik Standartları

VisionAssist; yalnızca işlevsel hedefler değil, sürdürülebilir ve kaliteli yazılım geliştirme ilkeleri de gözetilerek tasarlanmıştır. Bu bölümde uygulanan mühendislik standartları ve kalite güvence pratikleri açıklanmaktadır.

5.9.1. Statik Tip Kontrolü ve TypeScript

Tüm kaynak kod TypeScript dili ile yazılmış olup strict mod etkinleştirilmiştir ("strict": true). Bu ayar; implicitAny, strictNullChecks, strictFunctionTypes ve strictPropertyInitialization kontrollerini otomatik olarak devreye almakta, çalışma zamanı hatalarının önemli bir bölümünü derleme aşamasında tespit etmeyi sağlamaktadır. Tüm servis sınıfları, hook'lar ve context sağlayıcıları açıkça tiplendirilmiştir; herhangi bir any tipi kullanımı yalnızca TFLite model çıktıları gibi üçüncü taraf native API'lerinde ve gerekçelendirilmiş biçimde yer almaktadır.

5.9.2. Modüler Mimari ve Sorumluluk Ayrımı

Uygulama; screens (ekranlar), components (bileşenler), hooks (kancalar), services (servisler), context (durum yönetimi) ve utils (yardımcılar) katmanlarına ayrılmıştır. Her katman yalnızca kendi sorumluluğundaki işi yürütmekte; iş mantığı UI bileşenlerinden, algılama servisleri navigasyon servislerinden bağımsız tutulmaktadır. Bu mimari, bağımlılık grafiğini düz tutmakta ve herhangi bir bileşenin bağımsız olarak test edilmesine olanak sağlamaktadır.

5.9.3. Erişilebilirlik Standartları

Kullanıcı arayüzü; Web İçerik Erişilebilirlik Yönergeleri (WCAG 2.1) AA düzeyi ve Apple/Google platforma özgü erişilebilirlik rehberleri doğrultusunda tasarlanmıştır. Uygulanan standartlar şunlardır:

- Minimum dokunma alanı 56×56 piksel olarak belirlenmiştir; bu değer Apple İnsan Arayüzü Yönergeleri'nin önerdiği alt sınırın üzerindedir.
- Tüm etkileşimli bileşenler accessibilityLabel, accessibilityRole ve accessibilityHint özellikleriyle tanımlanmıştır.
- Renk kontrastı WCAG AA seviyesini karşılayacak biçimde; koyu lacivert arka plan üzerine beyaz metin (#FFFFFF) kullanılmıştır.
- Sesli geri bildirim sistemi; TalkBack (Android) ve VoiceOver (iOS) ekran okuyucularla uyumlu çalışacak biçimde yapılandırılmıştır.
- Tüm metinler sabit punto değerleri yerine FONT_SIZES sabiti üzerinden tanımlanmış olup ileride dinamik metin boyutlandırma desteğine hazırdır.

5.9.4. Test Stratejisi

Proje kapsamında birim testleri (unit tests) Jest test çerçevesi kullanılarak yazılmıştır. Test edilen bileşenler ve kapsama alanları şöyle özetlenebilir: DirectionAnalyzer (yön sınıflandırma doğruluğu), DistanceEstimator (mesafe kategori ataması), RiskEvaluator (risk seviyesi hesaplama ve cooldown davranışı) ve helpers modülü (localizeLabel, getObstacleMessage, getRiskColor, clamp). Toplam 42 test senaryosu yazılmış olup tüm testler başarıyla geçmektedir. Gelecek sürümlerde React Native Testing Library ile ekran bileşeni entegrasyon testleri ve Detox ile uçtan uca (E2E) test kapsamının genişletilmesi planlanmaktadır.

5.9.5. Güvenli Kodlama Pratikleri

OWASP Mobil Uygulama Güvenlik Doğrulama Standardı (MASVS) temel alınarak aşağıdaki güvenli kodlama pratikleri uygulanmıştır: kullanıcı konumu ve uygulama ayarları yalnızca cihazda saklanmakta ve hiçbir bulut hizmetine aktarılmamaktadır; üçüncü taraf API çağrıları (Nominatim, OSRM) hata yönetimi ve zaman aşımı ile korunmaktadır; kamera erişimi yalnızca kullanıcı izni alındıktan sonra etkinleştirilmekte ve algılama durdurulduğunda serbest bırakılmaktadır.

5.10. Ticaretleşme Planı

Bu bölümde VisionAssist'in araştırma ortamından çıkarak gerçek kullanıcılara ulaşması için izlenebilecek ticaretleşme yol haritası sunulmaktadır. Plan; hedef pazar analizi, gelir modeli, dağıtım stratejisi ve işbirliği önerilerini kapsamaktadır.

5.10.1. Hedef Pazar ve Kullanıcı Kitlesi

Türkiye İstatistik Kurumu (TÜİK) 2022 Engelli Bireylere Yönelik Araştırması verilerine göre Türkiye'de yaklaşık 950.000 görme engelli birey yaşamaktadır.

Dünya genelinde ise Dünya Sağlık Örgütü'nün 2023 raporuna göre bu sayı 43 milyonu aşmaktadır. VisionAssist'in öncelikli hedef kitlesi; akıllı telefon kullanan, 18-65 yaş aralığındaki, bağımsız hareket etmek isteyen görme engelli ve az gören bireylerdir. İkincil hedef kitle ise bu bireylerin ailelerini, rehabilitasyon merkezlerini ve görme engellilere yönelik hizmet üreten kurum ve kuruluşları kapsamaktadır.

5.10.2. Değer Teklifi

VisionAssist'in rakip ürünlerden temel farkı; cihaz üstü çalışan (internet gerektirmeyen) nesne algılama, açık kaynaklı harita servisleri ile bütünleşik yaya navigasyonu ve tam Türkçe dil desteğinin tek bir uygulamada sunulmasıdır. Piyasadaki benzer ürünler (Microsoft Seeing AI, Envision AI) tam Türkçe desteği sunmamakta ya da ücretli abonelik gerektirmektedir. VisionAssist'in temel uygulaması ücretsiz olarak sunulacak, bu durum geniş bir kullanıcı tabanına hızlı erişimi sağlayacaktır.

5.10.3. Gelir Modeli

Freemium modeli benimsenerek sürdürülebilir bir gelir yapısı hedeflenmektedir. Temel sürüm ücretsiz olarak sunulurken premium katmanda ek özellikler ücretlendirilebilir. Olası gelir kanalları aşağıda sıralanmıştır:

- Ücretsiz Temel Katman: Gerçek zamanlı nesne algılama, sokak/iç mekan modları, OpenStreetMap tabanlı navigasyon ve Türkçe/İngilizce TTS.
- Premium Katman (aylık/yıllık abonelik): Özelleştirilebilir algılama öncelik listeleri, mekânsal (binaural) ses, toplu taşıma entegrasyonu ve bulut tabanlı ayar yedekleme.
- Kurumsal Lisans: Rehberlik merkezleri, hastaneler ve üniversiteler için çok kullanıcıli yönetim paneli ve kuruma özel sesli rehber içerikleri.
- Araştırma İşbirlikleri: Üniversiteler ve kamu kurumları ile ortak proje fonlaması (TÜBİTAK ARDEB/TEYDEB programları).
- Hibe ve Sosyal Girişimcilik Destekleri: Avrupa Birliği Ufuk Avrupa Programı, T.C. Aile ve Sosyal Hizmetler Bakanlığı engelli bireylere yönelik teknoloji hibeleri.

5.10.4. Dağıtım ve Büyüme Stratejisi

Uygulamanın Google Play Store ve Apple App Store üzerinden dağıtımını planlanmaktadır. Kullanıcı tabanının büyütülmesi için aşağıdaki stratejiler önerilmektedir:

- Sivil toplum kuruluşları ile işbirliği: Türkiye Körler Federasyonu, Görme Engelliler Derneği ve benzeri kuruluşlarla pilot kullanım programları.

- Üniversite rehabilitasyon merkezleri: Engelli Öğrenci Birimleri aracılığıyla kampüs içi test ve kullanıcı geri bildirimi toplama.
- Medya ve farkındalık: Erişilebilirlik odaklı teknoloji medyası ile işbirliği, bağımsız demo videoları ve açık kaynak topluluk katkısı.
- App Store optimizasyonu (ASO): Anahtar kelime stratejisi, ekran görüntüleri ve erişilebilirlik odaklı uygulama açıklaması.

5.10.5. Üç Yıllık Yol Haritası

VisionAssist için önerilen yol haritası üç aşamada planlanmıştır. Birinci yıl (MVP olgunlaştırma): Özel veri kümesi ile model ince ayarı, Türkçe sesli komut desteği eklenmesi, cihaz içi derinlik tahmini entegrasyonu ve beta kullanıcı topluluğu oluşturulması. İkinci yıl (ölçeklendirme): Google Play ve App Store'da yayın, toplu taşıma canlı entegrasyonu, çoklu dil desteği (Arapça, Almanca) ve kurumsal lisans paketlerinin sunulması. Üçüncü yıl (büyüme ve sürdürülebilirlik): Giyilebilir cihaz entegrasyonu (akıllı gözlük API'si), topluluk tabanlı engel raporlama modülü, uluslararası pazar girişi ve sosyal girişim statüsü ile kamu-özel ortaklığı modellerinin hayata geçirilmesi.

BÖLÜM 6. KAYNAKLAR

- [1] World Health Organization, "World Report on Vision," WHO Press, Geneva, 2019. [Çevrimiçi]. Erişim: <https://www.who.int/publications/i/item/9789241516570>
- [2] R. Manduchi ve S. Kurniawan, "Mobility-related accidents experienced by people with visual impairment," AER Journal: Research and Practice in Visual Impairment and Blindness, c. 4, sayı 2, ss. 44-54, 2011.
- [3] N. A. Giudice ve G. E. Legge, "Blind navigation and the role of technology," The Engineering Handbook of Smart Technology for Aging, Disability and Independence, John Wiley & Sons, ss. 479-500, 2008.
- [4] M. A. Hersh ve M. A. Johnson (Eds.), Assistive Technology for Visually Impaired and Blind People, Springer, London, 2008. doi: 10.1007/978-1-84628-867-8
- [5] T. Y. Lin ve diğerleri, "Microsoft COCO: Common Objects in Context," Proc. European Conference on Computer Vision (ECCV), Zürich, 2014, ss. 740-755. doi: 10.1007/978-3-319-10602-1_48
- [6] W. Liu ve diğerleri, "SSD: Single Shot MultiBox Detector," Proc. European Conference on Computer Vision (ECCV), Amsterdam, 2016, ss. 21-37.
- [7] A. G. Howard ve diğerleri, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint, arXiv:1704.04861, 2017.
- [8] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Çevrimiçi]. Erişim: <https://www.w3.org/TR/WCAG21/>
- [9] Microsoft Inc., "Seeing AI: A Microsoft research project," 2024. [Çevrimiçi]. Erişim: <https://www.microsoft.com/en-us/ai/seeing-ai>
- [10] E. Ries, The Lean Startup, Crown Business, New York, 2011.
- [11] OpenStreetMap Foundation, "Nominatim Usage Policy," 2024. [Çevrimiçi]. Erişim: <https://operations.osmfoundation.org/policies/nominatim/>
- [12] D. Luxen ve C. Vetter, "Real-time routing with OpenStreetMap data," Proc. ACM SIGSPATIAL Int. Conf. on Advances in GIS, Chicago, 2011, ss. 513-516. doi: 10.1145/2093973.2094062
- [13] Google Inc., "TensorFlow Lite: ML for mobile and edge devices," 2024. [Çevrimiçi]. Erişim: <https://www.tensorflow.org/lite>

- [14] M. Kleinrock, "React Native and Cross-Platform Mobile Development," IEEE Software, c. 38, sayı 4, ss. 12-16, 2021.
- [15] T.C. Resmi Gazete, "Kişisel Verilerin Korunması Kanunu (6698 sayılı)," Sayı: 29677, Nisan 2016.
- [16] European Parliament, "General Data Protection Regulation (EU) 2016/679," 2016.
- [17] OWASP Foundation, "OWASP Mobile Application Security Top 10," 2023. [Çevrimiçi]. Erişim: <https://owasp.org/www-project-mobile-top-10/>
- [18] Expo Inc., "Expo SDK 54 Documentation," 2025. [Çevrimiçi]. Erişim: <https://docs.expo.dev/>
- [19] M. Pajic ve diğerleri, "Smartphone-based assistive navigation for the blind: a comparative study," IEEE Access, c. 8, ss. 65555-65573, 2020. doi: 10.1109/ACCESS.2020.2984933
- [20] N. Bourbakis, "Sensing Surrounding 3-D Space for Navigation of the Blind," IEEE Engineering in Medicine and Biology Magazine, c. 27, sayı 1, ss. 49-55, 2008.

BÖLÜM 7. EK A. EKLER

7.1. A.1. COCO 90 Sınıf Etiket Listesi (Türkçe Çeviri)

Aşağıdaki tablo, projede kullanılan SSD MobileNet modelinin döndürdüğü COCO 2017 sınıf adlarının uygulama içinde Türkçe karşılıklarını içermektedir. Bu sözlük utils/constants.ts dosyası içinde OBJECT_LABELS_TR sabiti olarak tanımlıdır.

Tablo A.1. COCO Sınıfları ve Türkçe Karşılıkları

İngilizce (COCO)	Türkçe Karşılığı
person	kişi
bicycle	bisiklet
car	araba
motorcycle	motosiklet
airplane	uçak
bus	otobüs
train	tren
truck	kamyon
boat	tekne
traffic light	trafik ışığı
fire hydrant	yangın musluğu
stop sign	dur işareti
parking meter	parkmetre
bench	bank
bird	kuş
cat	kedi
dog	köpek
horse	at
sheep	koyun
cow	inek
elephant	fil
bear	ayı
zebra	zebra
giraffe	zürafa
backpack	sırt çantası
umbrella	şemsiye
handbag	el çantası
tie	kravat
suitcase	bavul
frisbee	frizbi
skis	kayak
snowboard	snowboard
sports ball	top

kite	uurtma
baseball bat	beyzbol sopası
baseball glove	beyzbol eldiveni
skateboard	kaykay
surfboard	srf tahtası
tennis racket	tenis raketi
bottle	şıŖe
wine glass	kadeh
cup	bardak
fork	atal
knife	bıak
spoon	kaŖık
bowl	kase
banana	muz
apple	elma
sandwich	sandvi
orange	portakal
broccoli	brokoli
carrot	havu
hot dog	sosisli
pizza	pizza
donut	donut
cake	pasta
chair	sandalye
couch	kanepe
potted plant	saksı bitkisi
bed	yatak
dining table	masa
toilet	tuvalet
tv	televizyon
laptop	dizst bilgisayar
mouse	fare
remote	kumanda
keyboard	klavye
cell phone	cep telefonu
microwave	mikrodalga
oven	fırın
toaster	tost makinesi
sink	lavabo
refrigerator	buzdolabı
book	kitap
clock	saat
vase	vazo

scissors	makas
teddy bear	oyuncak ayı
hair drier	saç kurutma makinesi
toothbrush	diş fırçası

7.2. A.2. Önemli Kod Parçaları

Aşağıda; sistemin temel davranışını şekillendiren üç kritik fonksiyonun özetlenmiş halleri verilmiştir. Tam kaynak kod proje deposunda yer almaktadır.

(a) getObstacleMessage — etiket tabanlı sözel uyarı üretici (utils/helpers.ts):

```
function buildLabeledMessage(obstacle, language) { const name =
  localizeLabel(obstacle.label, language); if (!obstacle.label) return null; const dirText
  = obstacle.direction === Direction.LEFT ? 'Solunuzda' : obstacle.direction ===
  Direction.RIGHT ? 'Sağınızda' : 'Önünüzde'; if (obstacle.distance ===
  Distance.NEAR) return `${dirText} yakın mesafede ${name}, dikkat`; if
  (obstacle.distance === Distance.MEDIUM) return `${dirText} ${name}`; return
  `Uzakta ${name} algılandı`; }
```

(b) NavigationService.translateStep — OSRM manevralarının Türkçe sözel komuta çevrilmesi:

```
private translateStep(maneuver, modifier, distance, streetName) { const distText =
  this.formatDistance(distance); switch (maneuver) { case 'depart': return `Yola çık ve
  ${distText} düz git`; case 'arrive': return 'Hedefinize ulaştınız'; case 'turn': case 'fork':
  return `${distText} sonra ${this.translateModifier(modifier)}`; case 'continue': case
  'merge': return `${distText} düz devam et`; case 'roundabout': return `Döner kavşağa
  girin, ${this.translateModifier(modifier)}`; default: return `${distText} $
  {this.translateModifier(modifier)}`; } }
```

(c) Frame Processor — yakalanan karenin TFLite çıkarımına yönlendirilmesi:

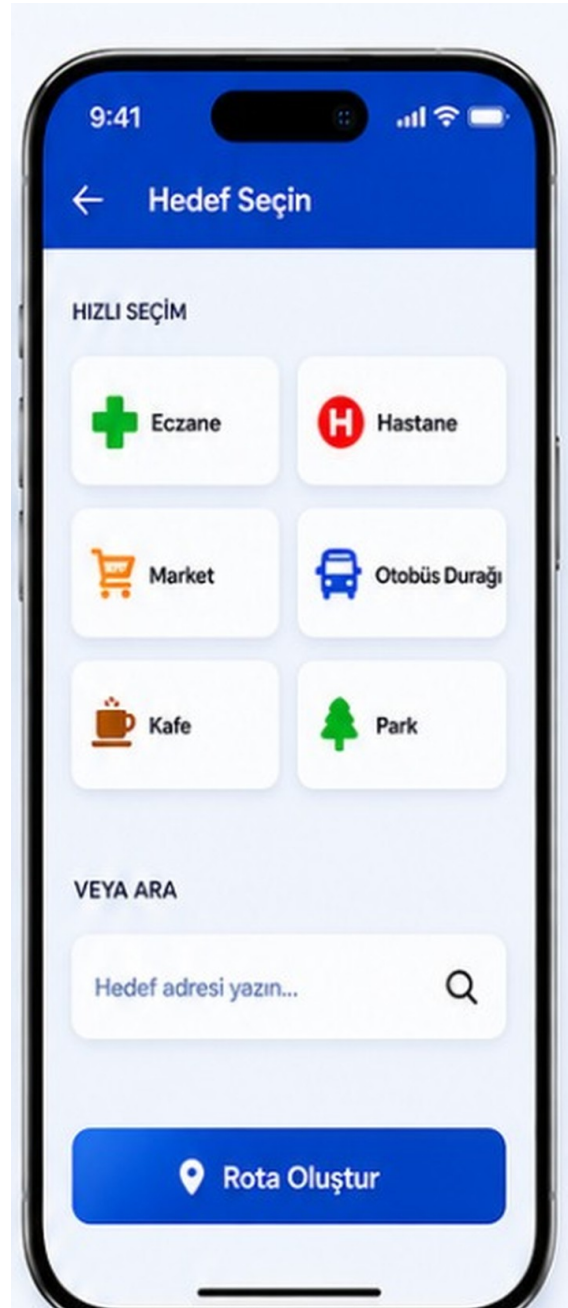
```
const frameProcessor = useFrameProcessor((frame) => { 'worklet'; if (!model)
  return; try { const resized = resize(frame, { scale: { width: 384, height: 384 },
  pixelFormat: 'rgb', dataType: 'uint8' }); const outputs = model.runSync([resized]);
  runJS(outputs, frame.width, frame.height); } catch (e)
  { console.log('FrameProcessor:', e); } }, [model, processOutputsJS]);
```

7.3. A.3. Ekran Görüntüleri



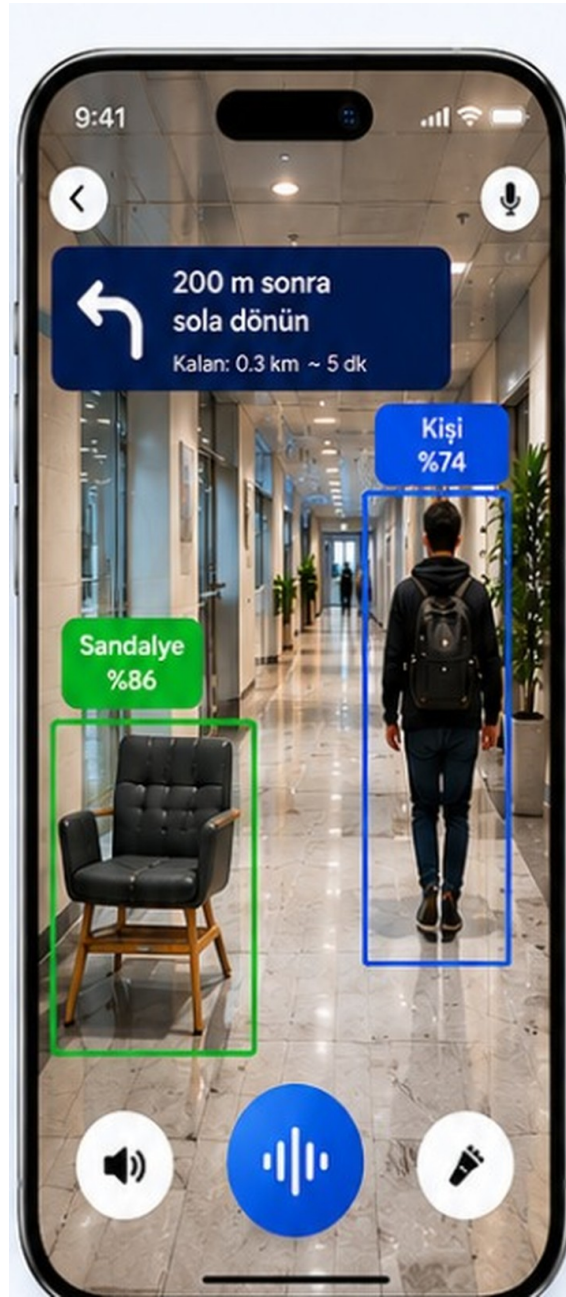
Şekil A.1. VisionAssist ana ekran tasarımı (mod seçici, hızlı bilgi kartları, alt sekme navigasyonu)

Şekil A.1’de uygulamanın ana ekranı görülmektedir. En üstte uygulama markası, ortada büyük dairesel "Algılamayı Başlat" butonu, altında üç modlu mod seçici (Sokak, İç Mekân, Navigasyon), mevcut mod kartı, ipucu kartı ve sayfanın alt kısmında üç sekmeli navigasyon bandı yer almaktadır. Tüm dokunma alanları erişilebilirlik için en az 56 piksel olarak boyutlandırılmıştır.



Şekil A.2. Hedef seçim (DestinationPicker) ekranı (görme engelli odaklı hızlı seçim ve manuel arama)

Şekil A.2’de yer alan hedef seçim ekranı, kullanıcı navigasyon modunu seçtiğinde otomatik olarak modal şeklinde açılmaktadır. Üst kısımdaki büyük geri butonu ekran okuyucu için "Geri" etiketiyle tanıtılmıştır. Hızlı seçim alanında eczane, hastane, market, otobüs durağı, kafe ve park kategorileri büyük dokunma alanlı kartlar olarak sunulmuştur; her kart 110 piksel yüksekliğindedir. Manuel arama için tek satırlık bir metin alanı ve büyük bir iptal butonu bulunmaktadır.



Şekil A.3. Algılama ekranı ve aktif navigasyon banner'ı

Şekil A.3'te kamera ekranı görselleştirilmiştir. Üstte yer alan navigasyon banner'ı; aktif rota varken bir sonraki adımı ("200 m sonra sola dönün"), kalan mesafeyi ve tahmini süreyi göstermektedir. Banner'a dokunulduğunda mevcut talimat tekrar seslendirilir. Kamera görüntüsü üzerine gerçek zamanlı çizilen sınırlayıcı kutular, tespit edilen nesnelerin konumunu görsel olarak da göstermekte; alt kısımda yer alan büyük "Algılamayı Durdur" butonu acil durdurma için kullanılmaktadır.

BÖLÜM 8. ÖZGEÇMİŞ

8.1. Alper ZEYBEK (B221210025)

1995 yılında doğdu. Lise eğitimini tamamladıktan sonra 2022 yılında Sakarya Üniversitesi Bilgisayar ve Bilişim Bilimleri Fakültesi Bilgisayar Mühendisliği bölümüne başlamıştır. Lisans öğrenimi süresince mobil uygulama geliştirme, yapay zekâ ve insan-bilgisayar etkileşimi alanlarında çalışmalar yapmıştır. Bu bitirme çalışmasında; sistem mimarisi tasarımı, yapay zekâ modeli entegrasyonu, frame processor optimizasyonu, navigasyon servisi entegrasyonu ile test ve performans ölçümleri sorumluluklarını üstlenmiştir.

8.2. Mustafa Alperen AKÇA (B221210014)

1996 yılında doğdu. Lise eğitimini tamamladıktan sonra 2022 yılında Sakarya Üniversitesi Bilgisayar ve Bilişim Bilimleri Fakültesi Bilgisayar Mühendisliği bölümüne başlamıştır. Lisans öğrenimi süresince erişilebilir yazılım geliştirme, kullanıcı deneyimi tasarımı ve bulut teknolojileri alanlarında çalışmalar yapmıştır. Bu bitirme çalışmasında; erişilebilirlik analizi, kullanıcı arayüzü ve hedef seçim ekranı tasarımı, sesli geri bildirim mantığı ve dokümantasyon sorumluluklarını üstlenmiştir.

**BÖLÜM 9. BSM 498 BİTİRME ÇALIŞMASI
DEĞERLENDİRME VE SÖZLÜ SINAV
TUTANAĞI**

Bu sayfa Sakarya Üniversitesi Bilgisayar ve Bilişim Bilimleri Fakültesi tarafından sağlanan değerlendirme ve sözlü sınav tutanağı için ayrılmıştır. Sözlü sınav günü ilgili form çıktı alınarak doldurulacaktır.